

Block 3: GUI and Database Connections in Python

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 9

Python GUI Frameworks

Button, text box, menu എന്നിവയുള്ള window-based application-കൾ Python-ൽ എങ്ങനെ ഉണ്ടാക്കാം എന്നാണ് ഈ unit പഠിപ്പിക്കുന്നത്. Tkinter, PyQt, wxPython, PySimpleGUI, Kivy എന്നീ അഞ്ച് framework-കളും, ഓരോന്നും എപ്പോൾ തിരഞ്ഞെടുക്കണം എന്നതും ഇവിടെ കാണാം.

GUI, GUI frameworks and event-driven programming

Tkinter: introduction and the first window

Tkinter widgets

Layout management: pack(), grid(), place()

Event handling in Tkinter: callbacks and bind()

Beyond basic widgets: ttk Treeview and Notebook

Object-oriented Tkinter design

Integrating Tkinter with a database (SQLite)

Threading in Tkinter; advantages and limitations

PyQt: introduction and basic structure

PyQt widgets and Qt Designer

Signals and slots

PyQt layouts, model-view, threads and storage

wxPython: introduction and a simple window

wxPython controls and event handling

wxPython sizers and dialog boxes

A multi-panel wxPython application

PySimpleGUI: layout and the event loop

PySimpleGUI: themes, popups and non-blocking patterns

Kivy: introduction and the App class

Kivy widgets, layouts, KV language and events

Kivy screens, styling and saving data

Comparison of Python GUI frameworks

GUI, GUI frameworks and event-driven programming

GUI (Graphical User Interface): an interface through which users interact with a program using visual elements such as windows, buttons, menus and text boxes instead of typed commands.

GUI framework: a library that provides ready-made widgets, layout managers and an event loop for building GUI applications.

Event-driven programming: a style in which the program waits in an event loop and runs callback functions when events such as clicks or key presses occur.

GUI (Graphical User Interface): command type ചെയ്യുന്നതിനു പകരം window, button, menu, text box പോലുള്ള visual element-കൾ വഴി user program-മായി ഇടപെടുന്ന interface.

GUI framework: GUI application ഉണ്ടാക്കാൻ ready-made widget-കൾ, layout manager-കൾ, event loop എന്നിവ നൽകുന്ന library.

Event-driven programming: program ഒരു event loop-ൽ കാത്തിരിക്കുകയും click, key press പോലുള്ള event-കൾ ഉണ്ടാകുമ്പോൾ callback function-കൾ run ചെയ്യുകയും ചെയ്യുന്ന രീതി.

A command-line program runs top to bottom and asks for input with `input()`. A GUI program instead builds a window, then hands control to an event loop that waits for the user. When the user clicks a button, the loop calls the function attached to that button. This unit covers five Python GUI frameworks: Tkinter, PyQt, wxPython, PySimpleGUI and Kivy.

മലയാളത്തിൽ

Command-line program ഒരു ക്യൂവിൽ നിൽക്കുന്നതുപോലെ മുകളിൽ നിന്ന് താഴോട്ട് ഓടിത്തീരും. GUI program ഒരു ചായക്കുടക്കാരനെപ്പോലെയാണ്: കുട തുറന്ന് (window ഉണ്ടാക്കി) customer വരുന്നതുവരെ കാത്തിരിക്കും, ആരെങ്കിലും "ഒരു ചായ" എന്ന് പറഞ്ഞാൽ (event) അതിനുള്ള പണി (callback) ചെയ്യും. ഈ കാത്തിരിപ്പാണ് event loop.

Every GUI program: create app/window → create widgets → arrange them (layout) → connect events to functions → start the event loop

Event loop names: Tkinter `mainloop()`, PyQt `app.exec()`, wxPython `app.MainLoop()`, PySimpleGUI `while True: window.read()`, Kivy `App().run()`

Tkinter: introduction and the first window

Tkinter: Python's built-in standard GUI library, based on the Tcl/Tk toolkit, used to create desktop windows, widgets and dialogs.

Root window: the main application window, created with `tk.Tk()`; all other widgets are placed inside it.

Tkinter: Tcl/Tk toolkit അടിസ്ഥാനമാക്കിയുള്ള, desktop window, widget, dialog എന്നിവ ഉണ്ടാക്കാനുള്ള Python-ന്റെ built-in standard GUI library.

Root window: `tk.Tk()` ഉപയോഗിച്ച് ഉണ്ടാക്കുന്ന main application window; മറ്റെല്ലാ widget-ഉം ഇതിനുള്ളിലാണ് വയ്ക്കുന്നത്.

Tkinter comes with Python, so no `pip install` is needed. It is lightweight, cross-platform (Windows, macOS, Linux), has a good set of widgets, a Canvas for drawing, and three layout managers. A Tkinter program always has the same skeleton: `import`, create root, add widgets, call `mainloop()`.

മലയാളത്തിൽ

Tkinter Python-നൊപ്പം തന്നെ വരുന്നതുകൊണ്ട് ഒന്നും `install` ചെയ്യേണ്ട. ആദ്യം `Tk()` കൊണ്ട് ഒരു ശൂന്യമായ window (root) ഉണ്ടാക്കുന്നു, അതിനുള്ളിൽ Label, Button കൈ വയ്ക്കുന്നു, അവസാനം `mainloop()` വിളിക്കുന്നു. `mainloop()` ഇല്ലെങ്കിൽ window ഒന്ന് മിന്നി ഉടനെ അടഞ്ഞുപോകും.

```
import tkinter as tk (Python 3; module name is lowercase)
root = tk.Tk() - create the main window
root.title("...") - window title; root.geometry("400x300") - width x height in pixels
widget.config(option=value) - change a widget option at run time
root.mainloop() - start the event loop
```

EXAMPLE (THE UNIT'S FIRST PROGRAM)

Create a 400×300 window titled "My First Program" with a label "Hello, Student!" and a button "Click Me" that changes the label to "Button Clicked!".

```
import tkinter as tk

root = tk.Tk()
root.title("My First Program")
root.geometry("400x300")

label1 = tk.Label(root, text="Hello, Student!", font=("Arial", 16))
label1.pack(pady=10)

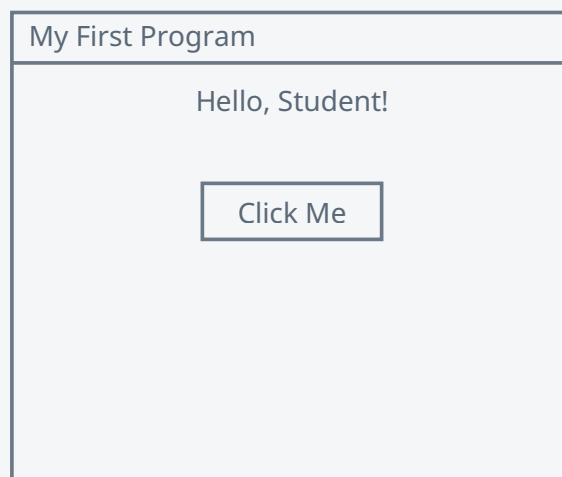
def on_click():
    label1.config(text="Button Clicked!")

newbutton = tk.Button(root, text="Click Me", command=on_click)
newbutton.pack(pady=40)

root.mainloop()
```

Step by step: `Tk()` makes the window; `title()` and `geometry()` set title and size; the Label is packed with 10 px vertical padding; the Button is packed with 40 px padding and its `command` is the function `on_click` (no brackets, because we pass the function, not call it); `mainloop()` keeps the window alive.

What you see: a window titled "My First Program" showing "Hello, Student!" near the top and a "Click Me" button below it. After one click the label reads "Button Clicked!".



Window layout before the click. After clicking, the label text becomes "Button Clicked!".

Exam tip: The unit's explanation says the button is packed "with vertical padding of 10 pixels", but the code uses `pady=40`; the code is what runs. Also remember: in Python 3 it is `import tkinter` (lowercase). `import Tkinter` is Python 2 and gives `ModuleNotFoundError` in Python 3.

Tkinter widgets

Widget: a GUI element such as a label, button or text box that the user sees or interacts with; in Tkinter it is created by instantiating a widget class with its parent container as the first argument.

Widget: user കാണുകയോ ഉപയോഗിക്കുകയോ ചെയ്യുന്ന label, button, text box പോലുള്ള GUI element; Tkinter-ൽ parent container ആദ്യ argument ആയി നൽകി widget class-ന്റെ object ഉണ്ടാക്കിയാണ് ഇത് സൃഷ്ടിക്കുന്നത്.

Every widget is created as `tk.WidgetName(parent, options...)`, where the parent is the root window or a Frame. Creating a widget does not show it; you must also call a layout method (`pack`, `grid` or `place`). Options like `text`, `font`, `width`, `bg` can be set at creation or later with `config()`.

മലയാളത്തിൽ

Widget-കൾ ഒരു വീട്ടിലെ furniture പോലെയാണ്: കസേര വാങ്ങിയാൽ മാത്രം പോരാ, അത് മുറിയിൽ എവിടെ വയ്ക്കണമെന്നും തീരുമാനിക്കണം. അതുപോലെ widget ഉണ്ടാക്കിയാലും `pack()`/`grid()`/`place()` വിളിച്ചാലേ window-ൽ കാണൂ.

Widget	Purpose
Label	Displays text or images
Button	Clickable button; runs its command
Entry	Single-line text input (<code>get()</code> , <code>delete(0, tk.END)</code>)
Text	Multi-line text input
Checkbutton	On/off checkbox
Radiobutton	One choice among several options
Listbox	List of items
Scale	Slider for a numeric value
Spinbox	Fixed-range input with up/down arrows
Canvas	Drawing lines, circles, shapes
Frame	Container that groups other widgets
Message	Like Label, but word-wraps long text
Menu	Menus for the window

EXAMPLE

Show a Checkbutton, two Radiobuttons and a Canvas with a circle; print the choices when a button is clicked.

```
import tkinter as tk

root = tk.Tk()
agree = tk.IntVar()
course = tk.StringVar(value="MCS-082")

tk.Checkbutton(root, text="I agree", variable=agree).pack()
tk.Radiobutton(root, text="MCS-082", variable=course, value="MCS-082").pack()
tk.Radiobutton(root, text="MCS-083", variable=course, value="MCS-083").pack()

canvas = tk.Canvas(root, width=120, height=80)
canvas.create_oval(35, 15, 85, 65)
canvas.pack()

tk.Button(root, text="Show",
          command=lambda: print(agree.get(), course.get())).pack()
root.mainloop()
```

Checkbuttons and Radiobuttons store their state in a Tkinter variable (IntVar, StringVar); read it with `.get()`. Radiobuttons sharing one variable form a group, so only one can be selected. If "I agree" is ticked and MCS-082 chosen, clicking Show prints 1 MCS-082 in the terminal.

Exam tip: For "draw shapes like lines and circles" the answer is **Canvas**; for "take text input" it is **Entry**; to change a label's text at run time use **config()** (also spelled **configure()**).

Layout management: pack(), grid(), place()

Layout (geometry) manager: the mechanism that decides the position and size of widgets inside a container. **pack()** stacks widgets in blocks, **grid()** places them in rows and columns, **place()** puts them at exact x/y coordinates.

Layout (geometry) manager: ഒരു container-നുള്ളിൽ widget-കളുടെ സ്ഥാനവും വലിപ്പവും തീരുമാനിക്കുന്ന സംവിധാനം. **pack()** widget-കളെ block ആയി അടുക്കുന്നു, **grid()** row-ഉം column-ഉം ആയി വയ്ക്കുന്നു, **place()** കൃത്യമായ x/y coordinate-ൽ വയ്ക്കുന്നു.

`pack()` is the simplest: widgets go one below another (default `side="top"`) or side by side with `side="left"`. `grid()` treats the container as a table, ideal for forms. `place()` gives exact pixel positions but does not adapt when the window is resized. Do not mix pack and grid in the same container.

മലയാളത്തിൽ

pack() ഒരു അലമാരയിൽ പുസ്തകം ഒന്നിനുമേൽ ഒന്നായി അടുക്കുന്നതുപോലെ. grid() ഒരു exam hall-ലെ seating chart പോലെ: row 2, column 3 എന്ന് പറഞ്ഞാൽ അവിടെ ഇരിക്കും. place() ചുമരിൽ ആണി അടിക്കുന്നതുപോലെ, കൃത്യം x, y-ൽ. ഒരേ Frame-ൽ pack-ഉം grid-ഉം ഒരുമിച്ച് ഉപയോഗിക്കരുത്.

```
w.pack(side="top" | "left" | "right" | "bottom", padx=, pady=, fill="x" | "y" | "both",
expand=True)
w.grid(row=r, column=c, padx=, pady=, sticky="w", columnspan=n)
w.place(x=px, y=py) or w.place(relx=0.5, rely=0.5, anchor="center")
```

EXAMPLE 1: GRID() FORM (THE UNIT'S EXAMPLE, COMPLETED)

```
import tkinter as tk

root = tk.Tk()
tk.Label(root, text="Username").grid(row=0, column=0, padx=5, pady=5)
tk.Entry(root).grid(row=0, column=1, padx=5, pady=5)
tk.Label(root, text="Password").grid(row=1, column=0, padx=5, pady=5)
tk.Entry(root, show="*").grid(row=1, column=1, padx=5, pady=5)
root.mainloop()
```

The labels sit in column 0 and the entry boxes in column 1, so everything lines up like a two-column table.



grid(): labels in column 0, Entry boxes in column 1.

EXAMPLE 2: PACK() SIDE BY SIDE AND PLACE()

```
import tkinter as tk

root = tk.Tk()
root.geometry("300x150")
tk.Button(root, text="Left").pack(side="left", padx=5)
tk.Button(root, text="Right").pack(side="right", padx=5)
tk.Label(root, text="Centre").place(relx=0.5, rely=0.5, anchor="center")
root.mainloop()
```

"Left" hugs the left edge, "Right" the right edge, and the label is placed exactly at the centre of the window (relx/rely are fractions of the window size, so it stays centred when resized).

Exam tip: "Difference between pack(), grid() and place()" is a standard question: pack = relative stacking (simplest), grid = rows and columns (forms), place = absolute coordinates (exact but not responsive). The unit's fill-in answer "the method used to place a widget with the default layout" is **pack()**.

Event handling in Tkinter: callbacks and bind()

Event: a user or system action, such as a mouse click or key press, that the GUI can respond to.

Callback: a function passed to a widget (e.g. through a Button's command option) that the event loop calls automatically when the event occurs.

Event: GUI-ക്ക് പ്രതികരിക്കാൻ കഴിയുന്ന mouse click, key press പോലുള്ള user-ന്റെയോ system-ന്റെയോ പ്രവൃത്തി.

Callback: event ഉണ്ടാകുമ്പോൾ event loop തനിയെ വിളിക്കുന്ന, widget-ന് (ഉദാ. Button-ന്റെ command option വഴി) കൈമാറുന്ന function.

The simplest event hook is `command=function_name` on a Button. Pass the function name without parentheses; writing `command=greet()` would call it once while building the window and attach its return value (None). For other events (keys, mouse, focus) use `widget.bind("<Event>", handler)`; the handler receives an event object.

മലയാളത്തിൽ

Callback എന്നാൽ "ഞാൻ ഇപ്പോൾ ചെയ്തില്ല, button അമർത്തുമ്പോൾ ഇത് ചെയ്യണം" എന്ന് ഏൽപ്പിച്ചു വരുന്ന function. അതുകൊണ്ട് `command=greet` എന്ന് പേര് മാത്രം കൊടുക്കണം, `greet()` എന്ന് bracket ഇട്ടാൽ അപ്പോൾ തന്നെ run ആകും. Enter key, mouse click പോലുള്ളവയ്ക്ക് `bind()` ഉപയോഗിക്കാം.

```
tk.Button(root, text="...", command=func) – func() takes no arguments
widget.bind("<Button-1>", handler) – left click; "<Return>" – Enter key; "<Key>" – any key
def handler(event): ... – event.x, event.y, event.char, event.widget
entry.get() – read the text in an Entry
```

EXAMPLE 1: BUTTON WITH CALLBACK (THE UNIT'S EXAMPLE, MADE COMPLETE)

```
import tkinter as tk

def greet():
    print("Hello User!")

root = tk.Tk()
btn = tk.Button(root, text="Welcome", command=greet)
btn.pack()
root.mainloop()
```

The window shows one button "Welcome". Each click prints Hello User! in the terminal (not in the window).

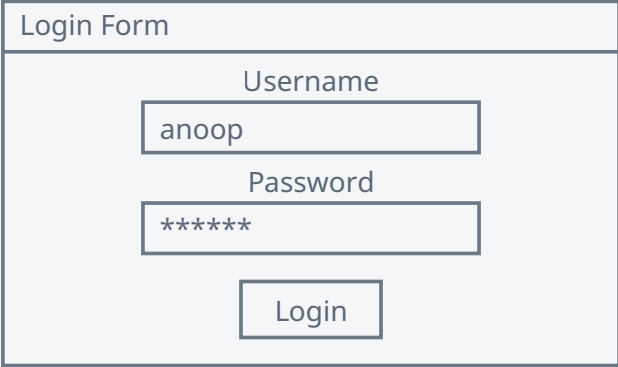
EXAMPLE 2: SIMPLE LOGIN FORM

```
import tkinter as tk

def login():
    user = username.get()
    pwd = password.get()
    print(f"Username: {user}, Password: {pwd}")

root = tk.Tk()
root.title("Login Form")
tk.Label(root, text="Username").pack()
username = tk.Entry(root)
username.pack()
tk.Label(root, text="Password").pack()
password = tk.Entry(root, show="*")
password.pack()
tk.Button(root, text="Login", command=login).pack(pady=10)
root.mainloop()
```

`show="*"` hides the password characters. Typing `anoop / abc123` and clicking Login prints `Username: anoop, Password: abc123` in the terminal.



The image shows a graphical user interface for a login form. The window has a title bar that says "Login Form". Inside the window, the widgets are arranged vertically. At the top is a label "Username". Below it is a text entry field containing the text "anoop". Below that is a label "Password". Below the password label is another text entry field, but its content is masked with seven asterisks "*****". At the bottom of the form is a button labeled "Login".

`pack()` stacks the five widgets vertically.

EXAMPLE 3: BIND() THE ENTER KEY

```
import tkinter as tk

root = tk.Tk()
entry = tk.Entry(root)
entry.pack()
out = tk.Label(root, text="")
out.pack()

def on_enter(event):
    out.config(text="You typed: " + entry.get())

entry.bind("<Return>", on_enter)
root.mainloop()
```

Typing Kochi and pressing Enter makes the label read "You typed: Kochi". Note that a `bind()` handler must accept the event parameter; a command callback must not.

Exam tip: The unit prints its "Button with Callback" and "Login Form" code with the function bodies un-indented (a PDF extraction problem); in real Python that is an `IndentationError`. Always indent function bodies. The Button's command attribute "executes a function on click".

Beyond basic widgets: ttk Treeview and Notebook

ttk: the themed-widget module `tkinter.ttk`, which provides modern-looking widgets such as Treeview, Notebook and Combobox.

Treeview: a ttk widget that displays hierarchical or tabular data in rows and columns. **Notebook:** a ttk widget that shows several pages (frames) as tabs in one window.

ttk: Treeview, Notebook, Combobox പോലുള്ള modern look ഉള്ള widget-കൾ നൽകുന്ന `tkinter.ttk` എന്ന themed-widget module.

Treeview: hierarchical അല്ലെങ്കിൽ tabular data row-ഉം column-ഉം ആയി കാണിക്കുന്ന ttk widget. **Notebook:** ഒരേ window-ൽ പല page-കൾ (frame-കൾ) tab ആയി കാണിക്കുന്ന ttk widget.

Treeview is used for tables: declare column ids, set headings, then `insert("", "end", values=(...))` adds a top-level row. `show="headings"` hides the empty tree column on the left. Notebook holds one Frame per tab; each frame is added with `notebook.add(frame, text="Tab name")`.

മലയാളത്തിൽ

Treeview ഒരു attendance register പോലെ: മുകളിൽ column heading, താഴെ ഓരോ student-ന്റെയും row. Notebook ഒരു browser-ലെ tab-കൾ പോലെ: Home tab, Settings tab, ഓരോന്നിലും വേറെ വേറെ content.

EXAMPLE 1: TREEVIEW TABLE (THE UNIT'S EXAMPLE)

```
from tkinter import ttk, Tk

root = Tk()
tree = ttk.Treeview(root, columns=("Name", "Age"), show="headings")
tree.heading("Name", text="Name")
tree.heading("Age", text="Age")
tree.insert("", "end", values=("Rohan", 24))
tree.insert("", "end", values=("Mohan", 27))
tree.pack()
root.mainloop()
```

The window shows a two-column table with headings Name and Age, and rows Rohan 24 and Mohan 27.

Name	Age
Rohan	24
Mohan	27

ttk.Treeview with show="headings".

EXAMPLE 2: NOTEBOOK WITH TWO TABS (CORRECTED)

```
from tkinter import ttk, Tk, Label

root = Tk()
notebook = ttk.Notebook(root)
frame1 = ttk.Frame(notebook)
frame2 = ttk.Frame(notebook)
Label(frame1, text="Home Page").pack(pady=20)
Label(frame2, text="Settings Page").pack(pady=20)
notebook.add(frame1, text="Home")
notebook.add(frame2, text="Settings")
notebook.pack(expand=True, fill="both")
root.mainloop()
```

The window shows two tabs, "Home" and "Settings"; clicking a tab shows "Home Page" or "Settings Page".

Exam tip: The unit's Notebook code uses frame2 without ever creating it, so it stops with `NameError: name 'frame2' is not defined`. The line `frame2 = ttk.Frame(notebook)` (added above) fixes it.

Object-oriented Tkinter design

Object-oriented GUI design: building the application as a class (e.g. a subclass of `tk.Tk` or `tk.Frame`) whose constructor creates the widgets and whose methods act as callbacks.

Object-oriented GUI design: application-നെ ഒരു class ആയി (ഉദാ. `tk.Tk` അല്ലെങ്കിൽ `tk.Frame`-ന്റെ subclass) എഴുതുന്ന രീതി; അതിന്റെ constructor widget-കൾ ഉണ്ടാക്കുന്നു, methods callback ആയി പ്രവർത്തിക്കുന്നു.

For large programs, putting the GUI in a class keeps widgets and their data together (as `self.xxx`), avoids global variables, and makes parts reusable. The class calls `super().__init__()` first so that the Tk window is created, then builds its widgets.

മലയാളത്തിൽ

ചെറിയ program-ൽ global variable മതി, പക്ഷേ വലിയ application-ൽ എല്ലാം ഒരു class-നുള്ളിൽ വെച്ചാൽ ഒരു വീട്ടിലെ സാധനങ്ങൾ ഓരോ മുറിയിൽ ഒതുക്കി വയ്ക്കുന്നതുപോലെ വൃത്തിയായി ഇരിക്കും. `super().__init__()` ആദ്യം വിളിക്കാൻ മറക്കരുത്, ഇല്ലെങ്കിൽ window തന്നെ ഉണ്ടാകില്ല.

EXAMPLE (THE UNIT'S OOP EXAMPLE)

```
import tkinter as tk

class MyApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("OOP Tkinter Example")
        self.geometry("300x200")
        tk.Button(self, text="Click Me",
                  command=self.show_message).pack(pady=40)

    def show_message(self):
        tk.Label(self, text="Hello, OOP World!").pack()

app = MyApp()
app.mainloop()
```

Because MyApp is a Tk window, self is used as the parent. Each click on "Click Me" packs a new label, so three clicks show "Hello, OOP World!" three times. To show it only once, create the label in __init__ with empty text and change it with config() in the callback.

Integrating Tkinter with a database (SQLite)

Persistent storage: saving application data to a file or database so that it survives after the program closes; with Tkinter this is commonly done through the built-in sqlite3 module.

Persistent storage: program അടച്ചതിനു ശേഷവും data നിലനിൽക്കാൻ അത് file-ലോ database-ലോ save ചെയ്യുന്നത്; Tkinter-ൽ ഇത് സാധാരണ built-in sqlite3 module വഴി ചെയ്യുന്നു.

A button's callback reads the Entry boxes, opens a connection, runs CREATE TABLE IF NOT EXISTS and a parameterised INSERT with ? placeholders, commits, closes, and clears the entries. Parameter placeholders prevent SQL injection.

മലയാളത്തിൽ

Entry-യിൽ type ചെയ്തത് window അടച്ചാൽ പോകും. അത് സൂക്ഷിക്കാൻ Save button അമർത്തുമ്പോൾ sqlite3 വഴി database file-ൽ എഴുതുന്നു, ഒരു register-ൽ പേര് എഴുതിവയ്ക്കുന്നതുപോലെ. commit() ചെയ്തില്ലെങ്കിൽ data save ആകില്ല.

EXAMPLE (THE UNIT'S SQLITE + TKINTER PROGRAM)

```
import tkinter as tk
import sqlite3

def save_data():
    conn = sqlite3.connect("students.db")
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS students (name TEXT, roll INTEGER)")
    c.execute("INSERT INTO students VALUES (?, ?)",
              (entry_name.get(), entry_roll.get()))
    conn.commit()
    conn.close()
    entry_name.delete(0, tk.END)
    entry_roll.delete(0, tk.END)

root = tk.Tk()
root.title("Database Integration")
tk.Label(root, text="Name").pack()
entry_name = tk.Entry(root)
entry_name.pack()
tk.Label(root, text="Roll").pack()
entry_roll = tk.Entry(root)
entry_roll.pack()
tk.Button(root, text="Save", command=save_data).pack(pady=10)
root.mainloop()
```

Window: labels Name and Roll, each with an entry box, and a Save button. After entering "Rohan", "12" and clicking Save, the row (Rohan, 12) is stored in `students.db` and both boxes become empty. `entry.delete(0, tk.END)` deletes from the first character to the end.

Threading in Tkinter; advantages and limitations

Thread: an independent flow of execution inside a program. In a GUI, long tasks run in a background thread so the main thread's event loop stays free and the window does not freeze.

Thread: ഒരു program-നുള്ളിലെ സ്വതന്ത്രമായ ഒരു execution flow. GUI-ൽ window freeze ആകാതിരിക്കാൻ നീണ്ട ജോലികൾ background thread-ൽ run ചെയ്യുന്നു, main thread-ലെ event loop free ആയി നിൽക്കും.

If a callback runs `time.sleep(5)` or a download in the main thread, `mainloop()` cannot process events and the window stops responding. Start the work with threading. `Thread(target=..., daemon=True).start()`. Tkinter widgets are not thread-safe, so the worker should hand results back (e.g. through a `queue.Queue`) and the main thread updates widgets, polling with `root.after()`.

മലയാളത്തിൽ

ഒരു ഹോട്ടലിലെ cashier (main thread) തന്നെ അടുക്കളയിൽ പോയി ബിരിയാണി ഉണ്ടാക്കാൻ നിന്നാൽ billing counter നിലയ്ക്കും. അതുകൊണ്ട് പാചകം cook-നെ (worker thread) ഏൽപ്പിക്കുന്നു; cook ready ആയാൽ counter-ൽ ഒരു ഭോക്കൺ (queue) വയ്ക്കും, cashier അത് നോക്കി customer-നോട് പറയും (widget update).

`threading.Thread(target=func, daemon=True).start()` – run func in background

`root.after(ms, func)` – call func on the main thread after ms milliseconds

`queue.Queue(), q.put(x), q.get_nowait()` – thread-safe hand-over

EXAMPLE (THE UNIT'S THREADING PROGRAM, MADE THREAD-SAFE)

```
import tkinter as tk
import threading, time, queue

q = queue.Queue()

def long_task():
    for i in range(5):
        time.sleep(1)
        q.put(f"Step {i+1} completed")

def start_task():
    threading.Thread(target=long_task, daemon=True).start()

def poll_queue():
    try:
        while True:
            label.config(text=q.get_nowait())
    except queue.Empty:
        pass
    root.after(100, poll_queue)

root = tk.Tk()
root.title("Threading Example")
label = tk.Label(root, text="Click Start", font=("Arial", 14))
label.pack(pady=20)
tk.Button(root, text="Start Task", command=start_task).pack()
poll_queue()
root.mainloop()
```

Window: "Click Start" and a "Start Task" button. After clicking, the label changes once a second: "Step 1 completed" ... "Step 5 completed", and the window can still be moved and resized during the task.

Advantages of Tkinter	Limitations of Tkinter
Easy to learn; good for beginners	Basic, dated appearance
No external dependency (ships with Python)	Limited native look and feel, especially on macOS
Large community, lots of documentation	Less suitable for complex, highly customised apps
Good for prototypes and small internal tools	

Exam tip: The unit's threading example calls `label.config()` directly from the worker thread. It often appears to work, but Tkinter should only be touched from the main thread; the `queue + after()` pattern above is the safe version. Also, the unit says "Tkinter supports threading for parallelism": Python threads give concurrency (the GUI stays responsive), not true CPU parallelism, because of the GIL.

PyQt: introduction and basic structure

PyQt: a set of Python bindings for the C++ Qt framework, used to build professional, cross-platform GUI applications (PyQt5 for Qt 5, PyQt6 for Qt 6).

QApplication: the object that initialises and manages a PyQt GUI application and runs its event loop; exactly one is created per program.

PyQt: professional, cross-platform GUI application ഉണ്ടാക്കാൻ ഉപയോഗിക്കുന്ന, C++ Qt framework-ന്റെ Python bindings (Qt 5-ന് PyQt5, Qt 6-ന് PyQt6).

QApplication: PyQt GUI application-നെ initialise ചെയ്ത് manage ചെയ്യുകയും event loop run ചെയ്യുകയും ചെയ്യുന്ന object; ഒരു program-ൽ ഒന്ന് മാത്രം ഉണ്ടാക്കുന്നു.

PyQt must be installed (`pip install PyQt6` or

`PyQt5`). Its features: cross-platform, a rich widget set (tables, tabs, trees, graphics views), custom widgets and themes (style sheets), and GPL/commercial licensing. A PyQt program follows six steps: import, create the `QApplication`, create widgets, set properties and layout, `show()` the window, and run the event loop with `app.exec()`.

മലയാളത്തിൽ

Tkinter ഒരു സാധാരണ സൈക്കിൾ ആണെങ്കിൽ PyQt ഒരു car പോലെ: install ചെയ്യണം, പഠിക്കാൻ കുറച്ച് കൂടുതൽ സമയം വേണം, പക്ഷേ professional app-ന് വേണ്ട എല്ലാ സൗകര്യവും ഉണ്ട്. ആദ്യം QApplication, പിന്നെ window, അവസാനം show()-ഉം exec()-ഉം.

```
app = QApplication(sys.argv) – create the application
```

```
QWidget() – base class of all windows and controls
```

```
window.setGeometry(x, y, width, height); button.move(x, y)
```

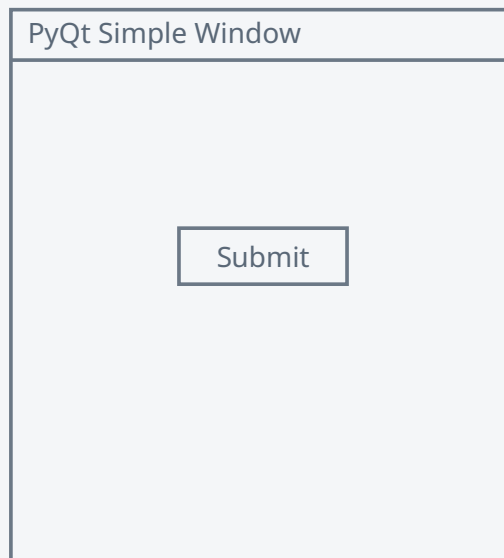
```
window.show() – make it visible; sys.exit(app.exec()) – run the loop and exit cleanly
```

EXAMPLE (THE UNIT'S BASIC PYQT6 WINDOW, CORRECTED)

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QPushButton

app = QApplication(sys.argv)
window = QWidget()
window.setWindowTitle("PyQt Simple Window")
window.setGeometry(300, 300, 300, 300) # x, y, width, height
button = QPushButton("Submit", window)
button.move(100, 100)
window.show()
sys.exit(app.exec())
```

A 300×300 window titled "PyQt Simple Window" appears at screen position (300, 300) with a "Submit" button whose top-left corner is at (100, 100) inside it. Passing window as the second argument makes the button a child of the window.



Button placed absolutely with move(100, 100).

Exam tip: Three slips in the unit here: (1) the last line `sys.exit(app.exec())` is missing a closing bracket (SyntaxError); (2) the explanation says `app.exec_()` starts the loop, but in PyQt6 only `app.exec()` exists (`exec_()` is the PyQt5 spelling); (3) the features list is headed "Key Features of Tkinter" but describes PyQt.

PyQt widgets and Qt Designer

Qt Designer: a drag-and-drop visual tool for designing Qt interfaces; it saves the layout as a .ui file, which pyuic5/pyuic6 converts into Python code.

Qt Designer: Qt interface drag-and-drop ആയി design ചെയ്യാനുള്ള visual tool; layout .ui file ആയി save ചെയ്യുന്നു, pyuic5/pyuic6 അതിനെ Python code ആക്കി മാറ്റുന്നു.

PyQt widget class names start with Q. Qt Designer separates design (the .ui file) from logic (your Python code), which speeds up development and reduces layout mistakes.

മലയാളത്തിൽ

Qt Designer ഒരു വീടിന്റെ plan വരയ്ക്കുന്ന software പോലെ: mouse കൊണ്ട് button, text box ഒക്കെ വലിച്ചിട്ട് design ചെയ്യാം. പിന്നെ pyuic6 ആ plan-നെ Python code ആക്കും, logic നമ്മൾ എഴുതിയാൽ മതി.

PyQt widget	Purpose	Tkinter equivalent
QLabel	Text or image	Label
QPushButton	Clickable button	Button
QLineEdit	Single-line input	Entry
QTextEdit	Multi-line editor	Text
QCheckBox	Checkbox	Checkbutton
QComboBox	Drop-down menu	ttk.Combobox
QListWidget	List of selectable items	Listbox
QTableWidget	Table of data	ttk.Treeview

Install tools (PyQt5): `pip install pyqt5-tools`

Convert: `pyuic5 form.ui -o form.py` (PyQt5) or `pyuic6 form.ui -o form.py` (PyQt6)

Exam tip: The MCQ "command used to convert .ui files into Python code" has answer **pyuic5** (the only option given). For PyQt6 the tool is pyuic6; the unit prints it as "pyuic6filename.ui" with the space missing.

Signals and slots

Signal: a notification a PyQt widget emits when an event happens, e.g. clicked for a button.

Slot: a function or method connected to a signal with `signal.connect(slot)`; it runs whenever the signal is emitted.

Signal: ഒരു event ഉണ്ടാകുമ്പോൾ PyQt widget പുറപ്പെടുവിക്കുന്ന അറിയിപ്പ്, ഉദാ. button-ന്റെ clicked.

Slot: `signal.connect(slot)` വഴി signal-മായി ബന്ധിപ്പിക്കുന്ന function അല്ലെങ്കിൽ method; signal വരുമ്പോഴെല്ലാം ഇത് run ആകും.

Signals and slots are PyQt's event-handling mechanism, the equivalent of Tkinter's `command=`. One signal can be connected to many slots and a slot can be any Python callable. Common signals: `clicked` (button), `textChanged` (QLineEdit), `currentIndexChanged` (QComboBox).

മലയാളത്തിൽ

Signal ഒരു doorbell അടിക്കുന്നതുപോലെ, slot ആ ശബ്ദം കേട്ട് വാതിൽ തുറക്കുന്ന ആൾ. `connect()` കൊണ്ട് bell-നെയും ആളെയും ബന്ധിപ്പിക്കുന്നു. ഒരു bell-ന് പല ആളുകളെ connect ചെയ്യാം.

```
widget.signal.connect(slot_function)
e.g. button.clicked.connect(my_function) (no brackets after my_function)
```

EXAMPLE (THE UNIT'S SIGNALS-AND-SLOTS PROGRAM)

```
from PyQt6.QtWidgets import QApplication, QPushButton

def on_click():
    print("Button clicked!")

app = QApplication([])
btn = QPushButton("Click Me")
btn.clicked.connect(on_click)
btn.show()
app.exec()
```

A button with no parent is shown as its own small window labelled "Click Me". Each click emits `clicked`, which calls the slot `on_click`, printing `Button clicked!` in the terminal.

Exam tip: For "describe a signal", pick "an event triggered by user interaction". In a 5-mark answer: define signal and slot, show one `connect()` line, and say the slot is called automatically when the signal is emitted.

PyQt layouts, model-view, threads and storage

Layout manager (PyQt): an object such as `QVBoxLayout`, `QHBoxLayout` or `QGridLayout` that automatically positions and resizes the widgets in a window.

Model-View architecture: a design that separates data storage (the model) from its display (the view), e.g. `QTableView` with a `QAbstractTableModel`.

Layout manager (PyQt): window-ലെ widget-കളുടെ സ്ഥാനവും വലിപ്പവും തനിയെ ക്രമീകരിക്കുന്ന `QVBoxLayout`, `QHBoxLayout`, `QGridLayout` പോലുള്ള object.

Model-View architecture: data സൂക്ഷിക്കുന്ന ഭാഗത്തെ (model) അത് കാണിക്കുന്ന ഭാഗത്തിൽ (view) നിന്ന് വേർതിരിക്കുന്ന design, ഉദാ. `QTableView`-ഉം `QAbstractTableModel`-ഉം.

Create a layout, `addWidget()` each widget (or `addLayout()` to nest layouts), then `window.setLayout(layout)`. Unlike `move()`, layouts adjust when the window is resized. For large data use `QTableView` + a model instead of `QTableWidget`. Long jobs go to `QThread` or `QRunnable` + `QThreadPool`. Data can be saved in JSON, SQLite or pickle files.

മലയാളത്തിൽ

`QVBoxLayout` ഒരു ടിഫിൻ box-ന്റെ തട്ടുകൾ പോലെ മുകളിൽ നിന്ന് താഴോട്ട്, `QHBoxLayout` ഒരു ബെഞ്ചിൽ ഇരിക്കുന്നവരെപ്പോലെ ഇടത്ത് നിന്ന് വലത്തോട്ട്, `QGridLayout` ഒരു table പോലെ. Model-View-ൽ data ഒരിടത്ത്, അതിന്റെ കാഴ്ച വേറൊരിടത്ത്, അതുകൊണ്ട് ഒരേ data പല രീതിയിൽ കാണിക്കാം.

`QVBoxLayout` – vertical; `QHBoxLayout` – horizontal; `QGridLayout` – rows × columns (`addWidget(w, row, col)`)

Threads: `QThread` (worker threads), `QRunnable` + `QThreadPool` (light tasks)

Storage: JSON, SQLite, pickle

EXAMPLE 1: QVBOXLAYOUT (THE UNIT'S EXAMPLE)

```
from PyQt6.QtWidgets import QVBoxLayout, QPushButton, QWidget, QApplication

app = QApplication([])
window = QWidget()
newlayout = QVBoxLayout()
newlayout.addWidget(QPushButton("One"))
newlayout.addWidget(QPushButton("Two"))
newlayout.addWidget(QPushButton("Three"))
window.setLayout(newlayout)
window.show()
app.exec()
```

Three full-width buttons One, Two, Three stacked top to bottom; if the window is widened, the buttons stretch with it.

EXAMPLE 2: SIMPLE FORM (CHECK YOUR PROGRESS 2, Q9)

Name field, Email field and Submit button; clicking prints the values in the console.

```
from PyQt6.QtWidgets import (QApplication, QWidget, QLabel, QLineEdit,
                             QPushButton, QVBoxLayout)

def on_submit():
    print(f"Name: {name_input.text()}")
    print(f"Email: {email_input.text()}")

app = QApplication([])
window = QWidget()
window.setWindowTitle("User Form")
layout = QVBoxLayout()
layout.addWidget(QLabel("Name:"))
name_input = QLineEdit()
layout.addWidget(name_input)
layout.addWidget(QLabel("Email:"))
email_input = QLineEdit()
layout.addWidget(email_input)
submit_button = QPushButton("Submit")
submit_button.clicked.connect(on_submit)
layout.addWidget(submit_button)
window.setLayout(layout)
window.show()
app.exec()
```

Entering "Asha" and "asha@example.com" and clicking Submit prints Name: Asha and Email: asha@example.com. `QLineEdit.text()` reads the typed text.

EXAMPLE 3: THE UNIT'S EXPENSE TRACKER (WHAT IT DOES)

The unit builds an `ExpenseTracker(QWidget)` class: a `QHBoxLayout` row with `QLineEdit`s for amount and note, a `QComboBox` for category and an "Add Expense" button; a filter `QComboBox` connected through `currentIndexChanged`; a 3-column `QTableWidget`; an embedded Matplotlib pie chart of spending per category; and a Save button that writes `expenses.json`. Invalid amounts are caught with `try/except ValueError` and shown with `QMessageBox.warning`.

Exam tip: The Expense Tracker imports `matplotlib.backends.backend_qt5agg` while using `PyQt6`. With `PyQt6` use `from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg`; mixing the Qt5 backend name with `PyQt6` is a common cause of import problems.

wxPython: introduction and a simple window

wxPython: a cross-platform Python GUI toolkit that wraps the C++ `wxWidgets` library and uses the operating system's native controls, so applications look native on each platform.

wx.Frame: the main top-level application window in wxPython. **wx.App:** the application object that initialises wxPython and runs the main loop.

wxPython: C++ `wxWidgets` library-യെ wrap ചെയ്യുന്ന, operating system-ന്റെ native control-കൾ ഉപയോഗിക്കുന്നതുകൊണ്ട് ഓരോ platform-ലും native ആയി തോന്നുന്ന cross-platform Python GUI toolkit.

wx.Frame: wxPython-ലെ main top-level application window. **wx.App:** wxPython initialise ചെയ്ത് main loop run ചെയ്യുന്ന application object.

Install with `pip install wxPython` (imported as `wx`). Features: cross-platform with native look, a large widget library (dialogs, panels, menus, grids), event-driven model and custom drawing. A minimal program has four steps: create the App, create a Frame, `Show()` it, start `MainLoop()`. Note wxPython method names start with a capital letter.

മലയാളത്തിൽ

wxPython-ൽ ഉണ്ടാക്കുന്ന app Windows-ൽ Windows app പോലെയും Mac-ൽ Mac app പോലെയും ഇരിക്കും, കാരണം അത് ആ OS-ന്റെ സ്വന്തം button-ഉം menu-ഉം ആണ് ഉപയോഗിക്കുന്നത്. ഓർക്കുക: `Show()`, `MainLoop()`, `Bind()`, എല്ലാം capital letter-ൽ തുടങ്ങും.

```
app = wx.App(False) – False: do not redirect print output to a popup window
frame = wx.Frame(None, title="...", size=(w, h)) – None = no parent
frame.Show(); app.MainLoop()
```

EXAMPLE (THE UNIT'S SIMPLE WINDOW)

```
import wx

app = wx.App(False)
frame1 = wx.Frame(None, title="Hello Student!", size=(200, 200))
frame1.Show()
app.MainLoop()
```

An empty 200×200 native window with the title "Hello Student!".

wxPython controls and event handling

Control: wxPython's name for a widget (e.g. wx.Button, wx.TextCtrl), placed inside a window such as a Frame or Panel.

Event binding (wxPython): linking an event type to a handler with `widget.Bind(wx.EVT_XXX, handler)`; the handler receives an event object.

Control: Frame അല്ലെങ്കിൽ Panel പോലുള്ള window-നുള്ളിൽ വയ്ക്കുന്ന widget-ന് (ഉദാ. wx.Button, wx.TextCtrl) wxPython-ൽ പറയുന്ന പേര്.

Event binding (wxPython): `widget.Bind(wx.EVT_XXX, handler)` ഉപയോഗിച്ച് ഒരു event type-നെ handler-മായി ബന്ധിപ്പിക്കുന്നത്; handler-ന് ഒരു event object കിട്ടും.

Controls are usually put on a `wx.Panel` inside the Frame (the panel gives the correct background and keyboard tab order). Common events: `wx.EVT_BUTTON` (button clicked), `wx.EVT_TEXT` (text changed), `wx.EVT_CLOSE` (window closing), `wx.EVT_MENU`, `wx.EVT_TIMER`.

മലയാളത്തിൽ

Frame ഒരു വീടിന്റെ ചുമതം മേൽക്കൂരയും, Panel അതിന്റെ തറ, control-കൾ ആ തറയിൽ വയ്ക്കുന്ന സാധനങ്ങൾ. Button click-ന് `wx.EVT_BUTTON`, text മാറ്റുമ്പോൾ `wx.EVT_TEXT`: ഇവ `Bind()` കൊണ്ട് function-മായി ബന്ധിപ്പിക്കുന്നു.

Control	Purpose
wx.StaticText	Displays read-only text
wx.TextCtrl	Text input field
wx.Button	Clickable button
wx.CheckBox / wx.RadioButton	Checkbox / radio button
wx.ListBox / wx.ComboBox	List of options / drop-down list
wx.Slider	Choose a value by sliding
wx.Notebook	Tabbed panels

EXAMPLE (THE UNIT'S "BUTTON AND EVENT" PROGRAM, SPACING RESTORED)

```
import wx

class MyfirstFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="Simple Button")
        panel1 = wx.Panel(self)
        self.button1 = wx.Button(panel1, label="Click Me", pos=(100, 50))
        self.button1.Bind(wx.EVT_BUTTON, self.on_click)

    def on_click(self, event):
        wx.MessageBox("You have clicked the button!", "Info",
                      wx.OK | wx.ICON_INFORMATION)

app = wx.App(False)
newframe = MyfirstFrame()
newframe.Show()
app.MainLoop()
```

A window "Simple Button" with a "Click Me" button at (100, 50). Clicking opens a message box titled "Info" with an information icon and the text "You have clicked the button!". The flags are combined with | (bitwise OR).

Exam tip: The unit's listing shows `import wx`, `class MyfirstFrame`, `def __init__`, `def on_click`: the spaces were lost in printing. Written that way it is a `SyntaxError`. Difference asked in exams: **wx.TextCtrl** accepts user input; **wx.StaticText** only displays read-only text.

wxPython sizers and dialog boxes

Sizer: wxPython's layout manager, which automatically arranges and resizes controls according to the window size (e.g. wx.BoxSizer, wx.GridSizer, wx.FlexGridSizer).

Dialog box: a small secondary window that shows a message or asks the user for input, such as a file or colour.

Sizer: window-ന്റെ വലിപ്പം അനുസരിച്ച് control-കളെ തനിയെ ക്രമീകരിക്കുകയും resize ചെയ്യുകയും ചെയ്യുന്ന wxPython layout manager (ഉദാ. wx.BoxSizer, wx.GridSizer, wx.FlexGridSizer).

Dialog box: ഒരു message കാണിക്കുകയോ file, colour പോലുള്ള input ചോദിക്കുകയോ ചെയ്യുന്ന ചെറിയ secondary window.

`wx.BoxSizer(wx.VERTICAL)` or `wx.HORIZONTAL` arranges controls in a column or a row. `sizer.Add(ctrl, proportion, flags, border)`: proportion 0 keeps natural size, 1 or more shares extra space; `wx.EXPAND` fills the cross direction; `wx.ALL` with a border adds space on all sides. Finish with `panel.SetSizer(sizer)`.

മലയാളത്തിൽ

Sizer ഒരു കല്യാണ പന്തലിലെ കസേര ക്രമീകരിക്കുന്ന ആളെപ്പോലെ: ഹാൾ വലുതായാലും ചെറുതായാലും അയാൾ കസേരകൾ അതിനനുസരിച്ച് നീക്കിവയ്ക്കും. proportion 1 കൊടുത്ത control അധിക സ്ഥലം എടുക്കും, 0 ആണെങ്കിൽ സ്വന്തം വലിപ്പത്തിൽ തന്നെ നിൽക്കും.

```
sizer.Add(ctrl, proportion, wx.EXPAND | wx.ALL, border)
```

`wx.BoxSizer` – row or column; `wx.GridSizer` – equal cells; `wx.FlexGridSizer` – rows/columns of different sizes

Dialogs: `wx.MessageBox`, `wx.FileDialog`, `wx.DirDialog`, `wx.ColourDialog`; `dlg.ShowModal()` == `wx.ID_OK`

EXAMPLE: VERTICAL LAYOUT WITH BOXSIZER

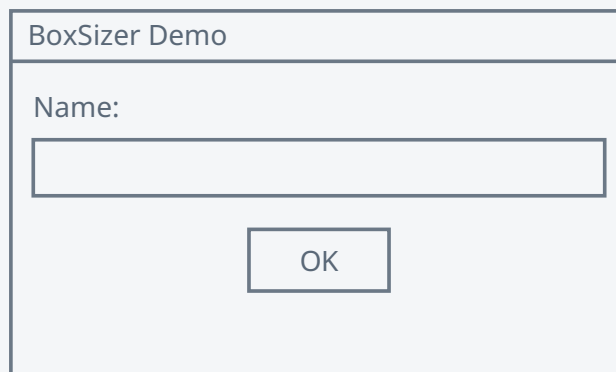
The unit shows only the output screen for "Vertical Layout using BoxSizer"; here is a matching program.

```
import wx

class VFrame(wx.Frame):
    def __init__(self):
        super().__init__(None, title="BoxSizer Demo", size=(250, 200))
        panel = wx.Panel(self)
        vbox = wx.BoxSizer(wx.VERTICAL)
        vbox.Add(wx.StaticText(panel, label="Name:"), 0, wx.ALL, 5)
        vbox.Add(wx.TextCtrl(panel), 0, wx.EXPAND | wx.ALL, 5)
        vbox.Add(wx.Button(panel, label="OK"), 0, wx.ALIGN_CENTER | wx.ALL, 5)
        panel.SetSizer(vbox)

app = wx.App(False)
VFrame().Show()
app.MainLoop()
```

Window: "Name:" at the top, a text box stretched across the width, and a centred OK button, each with a 5 px border. Widening the window widens only the text box.



`wx.BoxSizer(wx.VERTICAL)`: the `TextCtrl` uses `wx.EXPAND`, the button `wx.ALIGN_CENTER`.

EXAMPLE: MESSAGE DIALOG

```
wx.MessageBox("Operation completed", "Info", wx.OK | wx.ICON_INFORMATION)
```

Shows a modal box titled "Info" with the text "Operation completed" and an OK button. (It needs a running `wx.App`.)

Exam tip: "Which layout manager arranges widgets in a column or row?" → **wx.BoxSizer**.
"How does a sizer help?" → it arranges and resizes controls automatically as the window size changes, giving a responsive layout without hard-coded positions.

A multi-panel wxPython application

wx.Panel: a container window placed inside a Frame to hold and group controls; an app can be split into several reusable Panel subclasses.

wx.Timer: an object that fires `wx.EVT_TIMER` events at a fixed interval, used for clocks, auto-save and periodic updates.

wx.Panel: control-കളെ ഉൾക്കൊള്ളാനും group ചെയ്യാനും Frame-നുള്ളിൽ വയ്ക്കുന്ന container window; ഒരു app-നെ പല reusable Panel subclass-കളായി വിഭജിക്കാം.

wx.Timer: നിശ്ചിത ഇടവേളയിൽ `wx.EVT_TIMER` event ഉണ്ടാക്കുന്ന object; clock, auto-save, periodic update എന്നിവയ്ക്ക് ഉപയോഗിക്കുന്നു.

The unit's intermediate app is a to-do list split into parts: InputPanel (a TextCtrl + "Add" button in a horizontal sizer), ListPanel (a `wx.ListCtrl` in report style with columns ID, Task, Status), and MainFrame that stacks both in a vertical sizer, binds the Add button, and shows a status bar. A File menu (Save, Load, Exit with Ctrl shortcuts), `wx.FileDialog` for saving/loading JSON, and a 1-second `wx.Timer` clock complete it. A `wx.App` subclass creates everything in `OnInit()`, which must return `True`.

മലയാളത്തിൽ

ഒരു വലിയ app മുഴുവൻ ഒരു Frame-ൽ കുത്തിനിറയ്ക്കാതെ ഓരോ ഭാഗവും വേറെ Panel ആക്കുന്നു, ഒരു വീട്ടിൽ അടുക്കള, ഹാൾ, bedroom വേറെ വേറെ ഉള്ളതുപോലെ. ഇങ്ങനെ ചെയ്താൽ test ചെയ്യാനും വീണ്ടും ഉപയോഗിക്കാനും എളുപ്പം.

```
Menu: menubar = wx.MenuBar(), m = wx.Menu(), item = m.Append(wx.ID_SAVE, "&Save\tCtrl+S"),
menubar.Append(m, "&File"), self.SetMenuBar(menubar), self.Bind(wx.EVT_MENU, handler,
item)
ListCtrl: InsertColumn(i, "Head", width=), pos = InsertItem(index, text), SetItem(pos, col,
text)
Timer: self.timer = wx.Timer(self), self.Bind(wx.EVT_TIMER, handler, self.timer),
self.timer.Start(1000)
```

EXAMPLE (CONDENSED, RUNNABLE VERSION OF THE UNIT'S APP)

```

import wx

class InputPanel(wx.Panel):
    def __init__(self, parent):
        super().__init__(parent)
        sizer = wx.BoxSizer(wx.HORIZONTAL)
        self.txt = wx.TextCtrl(self)
        self.btn = wx.Button(self, label="Add")
        sizer.Add(self.txt, 1, wx.EXPAND | wx.ALL, 5)
        sizer.Add(self.btn, 0, wx.ALL, 5)
        self.SetSizer(sizer)

class ListPanel(wx.Panel):
    def __init__(self, parent):
        super().__init__(parent)
        sizer = wx.BoxSizer(wx.VERTICAL)
        self.list = wx.ListCtrl(self, style=wx.LC_REPORT)
        self.list.InsertColumn(0, "ID", width=60)
        self.list.InsertColumn(1, "Task", width=250)
        self.list.InsertColumn(2, "Status", width=100)
        sizer.Add(self.list, 1, wx.EXPAND | wx.ALL, 5)
        self.SetSizer(sizer)

    def add_item(self, idx, task, status="Pending"):
        pos = self.list.InsertItem(self.list.GetItemCount(), str(idx))
        self.list.SetItem(pos, 1, task)
        self.list.SetItem(pos, 2, status)

class MainFrame(wx.Frame):
    def __init__(self):
        super().__init__(None, title="Intermediate wxPython App", size=(600, 400))
        panel = wx.Panel(self)
        vbox = wx.BoxSizer(wx.VERTICAL)
        self.input_panel = InputPanel(panel)
        self.list_panel = ListPanel(panel)
        vbox.Add(self.input_panel, 0, wx.EXPAND)
        vbox.Add(self.list_panel, 1, wx.EXPAND)
        panel.SetSizer(vbox)
        self.input_panel.btn.Bind(wx.EVT_BUTTON, self.on_add)
        self.statusbar = self.CreateStatusBar()
        self.counter = 1
        self.timer = wx.Timer(self)
        self.Bind(wx.EVT_TIMER, self.on_tick, self.timer)
        self.timer.Start(1000)

    def on_add(self, event):
        text = self.input_panel.txt.GetValue().strip()
        if text:
            self.list_panel.add_item(self.counter, text)
            self.input_panel.txt.Clear()
            self.counter += 1

```

```

def on_tick(self, event):
    self.statusbar.SetStatusText(f"Clock: {wx.DateTime.Now().FormatISOTime()}")

class MyApp(wx.App):
    def OnInit(self):
        MainFrame().Show()
        return True

if __name__ == "__main__":
    MyApp(False).MainLoop()

```

Window: a text box with an "Add" button across the top, a table with columns ID / Task / Status filling the rest, and a status bar showing a live clock. Typing "Pay fees" and clicking Add inserts row 1 | Pay fees | Pending.

Intermediate wxPython App		
		Add
ID	Task	Status
1	Pay fees	Pending
Clock: 10:15:02		

InputPanel (top) and ListPanel (middle) in a vertical sizer, with a status bar.

Exam tip: In the unit's printed code the methods `_build_menu`, `on_save`, `on_load` and `start_timer` appear outside `MainFrame` with broken indentation, and `import json` sits between methods. They must be indented as methods of `MainFrame`, with `import json` at the top of the file.

PySimpleGUI: layout and the event loop

PySimpleGUI: a Python library that simplifies GUI programming by providing a simple wrapper over toolkits such as Tkinter, Qt, wxPython and Remi.

Layout (PySimpleGUI): a list of lists in which each inner list is one row of elements in the window.

PySimpleGUI: Tkinter, Qt, wxPython, Remi പോലുള്ള toolkit-കൾക്കു മുകളിൽ ഒരു ലളിതമായ wrapper നൽകി GUI programming എളുപ്പമാക്കുന്ന Python library.

Layout (PySimpleGUI): ഓരോ inner list-ഉം window-ലെ ഒരു row ആയി വരുന്ന list of lists.

You describe the window as rows of elements, create `sg.Window(title, layout)`, and write the event loop yourself: `event, values = window.read()` waits for an event and returns the event name (usually the button text or key) and a dictionary of input values. Break out on `sg.WINDOW_CLOSED` and finally call `window.close()`. It is ideal for quick prototypes and small tools.

മലയാളത്തിൽ

PySimpleGUI-ൽ window ഒരു ക്ലാസ് timetable പോലെ എഴുതുന്നു: ഓരോ വരിയും ഒരു list. പിന്നെ while True loop-ൽ `window.read()` "എന്ത് സംഭവിച്ചു?" എന്ന് ചോദിക്കും, ഉത്തരമായി event-ഉം values-ഉം കിട്ടും. Tkinter-ലെ callback-കൾക്കു പകരം നമ്മൾ തന്നെ loop എഴുതുന്നു, അതാണ് വ്യത്യാസം.

Elements: `sg.Text`, `sg.InputText/sg.Input`, `sg.Button`, `sg.Checkbox`, `sg.Radio`, `sg.Slider`, `sg.Combo`, `sg.Listbox`, `sg.Image`, `sg.ProgressBar`, `sg.Multiline`
`event, values = window.read(); window["-KEY-"].update(new_value); window.close()`

EXAMPLE 1 (THE UNIT'S BASIC APPLICATION)

```
import PySimpleGUI as sg

layout1 = [
    [sg.Text("Enter your first name:"), sg.InputText()],
    [sg.Button("OK"), sg.Button("Cancel")]
]

window = sg.Window("Basic GUI", layout1)
while True:
    event, values = window.read()
    if event == sg.WINDOW_CLOSED or event == "Cancel":
        break
    print("Hello Student!," , values[0])
window.close()
```

Row 1: a prompt and an input box; row 2: OK and Cancel. Typing "Meera" and clicking OK prints Hello Student!, Meera. The input had no key, so PySimpleGUI numbers it automatically: values[0].



Basic GUI	
Enter your first name:	
OK	Cancel

Each inner list of the layout is one row.

EXAMPLE 2 (CHECK YOUR PROGRESS 4, Q9, USING FREESIMPLEGUI)

An input field and a button; on click, the text appears beneath.

```
import FreeSimpleGUI as sg

layout = [
    [sg.Input(key="-IN-")],
    [sg.Button("Show")],
    [sg.Text("", size=(30, 1), key="-OUT-")]
]
window = sg.Window("Simple Text Viewer", layout)
while True:
    event, values = window.read()
    if event == sg.WINDOW_CLOSED:
        break
    if event == "Show":
        window["-OUT-"].update(values["-IN-"])
window.close()
```

Typing "Thrissur" and clicking Show makes "Thrissur" appear in the text element below the button. Giving elements keys like "-IN-" makes the code clearer than numeric indexes.

Exam tip: PySimpleGUI version 5 became a paid, registered product, so `pip install PySimpleGUI` may not give a freely usable version. The unit's note is right: install the open-source fork with `python -m pip install FreeSimpleGUI` and write `import FreeSimpleGUI as sg`; the rest of the code is unchanged. In the MCQ, the window is displayed (and events read) by **window.read()**.

PySimpleGUI: themes, popups and non-blocking patterns

Theme: a named set of colours and fonts applied to all PySimpleGUI windows with `sg.theme("Name")`.

Non-blocking event loop: an event loop that calls `window.read(timeout=ms)` so it returns periodically, letting the program process results from background threads without freezing the window.

Theme: `sg.theme("Name")` വഴി എല്ലാ PySimpleGUI window-കൾക്കും നൽകുന്ന colour-ഉം font-ഉം അടങ്ങിയ പേരുള്ള ഒരു set.

Non-blocking event loop: `window.read(timeout=ms)` വിളിക്കുന്നതിനാൽ ഇടയ്ക്കിടെ return ചെയ്യുന്ന event loop; window freeze ആകാതെ background thread-ന്റെ result-കൾ process ചെയ്യാൻ ഇത് സഹായിക്കുന്നു.

Themes: `sg.theme("DarkBlue")`; `sg.theme_previewer()` shows all themes. Popups: `sg.popup()`, `sg.popup_yes_no()`, `sg.popup_ok_cancel()`, `sg.popup_error()`, `sg.popup_get_text()`, `sg.popup_get_file()`. Never block the event loop: run slow work in a thread (or a `ThreadPoolExecutor`, or an `asyncio` loop inside a thread), put progress messages in a queue. Queue, and drain the queue after each `read(timeout=...)`. Other practical advice from the unit: build reusable "composite elements" as functions that return layout rows, update big tables in batches, edit table rows through a popup, save data with `json/csv` or `sqlite3`, load plugins with `pkgutil/importlib`, and support accessibility (tooltips, keyboard shortcuts, readable sizes).

മലയാളത്തിൽ

`window.read()` `timeout` ഇല്ലാതെ വിളിച്ചാൽ user എന്തെങ്കിലും ചെയ്യുന്നതുവരെ അവിടെ തന്നെ കാത്തുനിൽക്കും. `timeout=100` കൊടുത്താൽ ഓരോ 100 ms-ലും തിരിച്ചുവരും, അപ്പോൾ background thread queue-ൽ ഇട്ട message നോക്കി progress bar update ചെയ്യും. Postman ഇടയിലെ letter box നോക്കുന്നതുപോലെ.

EXAMPLE: PROGRESS BAR DRIVEN BY A WORKER THREAD

```
import threading, queue, time
import FreeSimpleGUI as sg

ui_queue = queue.Queue()

def worker():
    for i in range(1, 6):
        time.sleep(1)
        ui_queue.put(("progress", i * 20))
        ui_queue.put(("done", "Task completed"))

layout = [[sg.ProgressBar(100, orientation="h", size=(30, 20), key="-PROG-")],
          [sg.Button("Start"), sg.Button("Exit")]]
window = sg.Window("Worker Demo", layout)

while True:
    event, values = window.read(timeout=100)
    if event in (sg.WINDOW_CLOSED, "Exit"):
        break
    if event == "Start":
        threading.Thread(target=worker, daemon=True).start()
    try:
        while True:
            kind, data = ui_queue.get_nowait()
            if kind == "progress":
                window["-PROG-"].update(data)
            elif kind == "done":
                sg.popup(data)
    except queue.Empty:
        pass
window.close()
```

After Start, the bar fills 20 %, 40 %, ... 100 % over five seconds, then a popup says "Task completed". Only the main loop touches the window; the worker only writes to the queue. The unit's asyncio version is the same idea with `asyncio.run(task())` and `await asyncio.sleep(1)` inside the worker thread.

EXAMPLE: REUSABLE COMPOSITE ELEMENT (THE UNIT'S)

```
def LabeledInput(key, label, width=30):
    return [sg.Text(label), sg.Input(key=key, size=(width, 1))]

layout = [LabeledInput("-NAME-", "Name"), LabeledInput("-CITY-", "City")]
```

Each call returns one ready-made row, so a form with many labelled inputs stays short.

Advantages of PySimpleGUI	Limitations of PySimpleGUI
Very easy; least code for a window	Less flexible than using Tkinter/PyQt directly
Good documentation and cookbook	Limited low-level control of widgets
Fast prototyping; built-in themes	Advanced features need the underlying framework
Cross-platform	Tables are not directly editable

Kivy: introduction and the App class

Kivy: a free, open-source Python framework for building multi-touch, GPU-accelerated applications that run on Windows, Linux, macOS, Android and iOS from the same code.

App class (Kivy): the base class of every Kivy application; its `build()` method returns the root widget, and `run()` opens the window and starts the main loop.

Kivy: ഒരു code ഉപയോഗിച്ച് Windows, Linux, macOS, Android, iOS എന്നിവയിൽ run ചെയ്യുന്ന multi-touch, GPU-accelerated application-കൾ ഉണ്ടാക്കാനുള്ള free, open-source Python framework.

App class (Kivy): എല്ലാ Kivy application-ന്റെയും base class; അതിന്റെ `build()` method root widget return ചെയ്യുന്നു, `run()` window തുറന്ന് main loop തുടങ്ങുന്നു.

Kivy draws its own widgets with OpenGL, supports touch gestures and multi-touch, and lets you write the interface declaratively in the KV language. Install with `pip install kivy`. You subclass App, override `build()` to return the root widget, and call `MyApp().run()`.

മലയാളത്തിൽ

Mobile phone-യും computer-യും ഒരേ code-ൽ ഓടുന്ന app വേണമെങ്കിൽ Kivy. അത് OS-ന്റെ button ഉപയോഗിക്കുന്നില്ല, GPU ഉപയോഗിച്ച് സ്വന്തമായി വരയ്ക്കുന്നു, അതുകൊണ്ട് touch, swipe ഒക്കെ എളുപ്പം. `build()` "എന്റെ app-ൽ ആദ്യം എന്ത് കാണിക്കണം" എന്ന് പറയുന്ന method ആണ്.

```
from kivy.app import App
class MyApp(App): def build(self): return root_widget
MyApp().run() - note the class name MyApp becomes the window title "My"
```

EXAMPLE (THE UNIT'S BASIC KIVY APP, CORRECTED)

```
from kivy.app import App
from kivy.uix.label import Label

class MyApp(App):
    def build(self):
        return Label(text="Hello, Student!")

if __name__ == "__main__":
    MyApp().run()
```

A window (default 800×600, black background) with "Hello, Student!" in the centre; the Label fills the whole window and centres its text.

Exam tip: The unit writes `from kivy.app import NewApp` and `class MyApp(NewApp)`. There is no class `NewApp` in Kivy, so this raises `ImportError`. The correct base class is `App`. MCQ answers: the method overridden to build the UI is **build()** (the unit's key prints "Build()", but Python is case-sensitive); `run()` starts the main loop and shows the interface.

Kivy widgets, layouts, KV language and events

KV language: Kivy's declarative domain-specific language for describing the widget tree and its properties, keeping the presentation separate from the Python logic.

Widget (Kivy): a visual element such as Label, Button, TextInput or Slider that the user sees or interacts with; layouts are also widgets that contain other widgets.

KV language: widget tree-യും അതിന്റെ property-കളും വിവരിക്കാനുള്ള Kivy-യുടെ declarative domain-specific language; presentation-നെ Python logic-ൽ നിന്ന് വേർതിരിച്ചു നിർത്തുന്നു.

Widget (Kivy): user കാണുകയോ ഉപയോഗിക്കുകയോ ചെയ്യുന്ന Label, Button, TextInput, Slider പോലുള്ള visual element; layout-കളും മറ്റ് widget-കളെ ഉൾക്കൊള്ളുന്ന widget-കളാണ്.

Kivy widgets include Label, Button, TextInput, Image, Slider, Switch, CheckBox and ProgressBar. Layouts arrange children: BoxLayout (row or column), GridLayout (grid), FloatLayout (absolute/relative positions), AnchorLayout (anchor to an edge or centre). Events are bound with `widget.bind(on_press=callback)`. Lifecycle hooks: `build()`, `on_start()`, `on_stop()`, `on_pause()`, `on_resume()`.

മലയാളത്തിൽ

KV language ഒരു പാചകക്കുറിപ്പ് പോലെയാണ്: "BoxLayout-ന്റെ ഉള്ളിൽ ഒരു Label, അതിന് താഴെ ഒരു Button" എന്ന് indentation കൊണ്ട് വിവരിക്കുന്നു. Python file-ൽ logic മാത്രം. Label വെറുതെ text കാണിക്കും, TextInput-ൽ user type ചെയ്യും, ഇതാണ് ഇവ തമ്മിലുള്ള വ്യത്യാസം.

Kivy layout	Arrangement
BoxLayout	Widgets in a row or a column (orientation="vertical")
GridLayout	Grid of cells (cols= or rows=)
FloatLayout	Positions by pos_hint / size_hint (absolute-style)
AnchorLayout	Anchors children to top, bottom, left, right or centre

EXAMPLE: EVENT BINDING IN PYTHON CODE

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.label import Label

class ClickApp(App):
    def build(self):
        box = BoxLayout(orientation="vertical")
        self.lbl = Label(text="Not clicked")
        btn1 = Button(text="Submit")
        btn1.bind(on_press=self.callback_function)
        box.add_widget(self.lbl)
        box.add_widget(btn1)
        return box

    def callback_function(self, instance):
        self.lbl.text = "Submitted!"

ClickApp().run()
```

The window is split into two halves: the label on top, the Submit button below. Pressing Submit changes the label to "Submitted!". The callback receives the button that fired the event as instance.

Kivy screens, styling and saving data

ScreenManager: a Kivy widget that holds several named Screen objects and shows one at a time; setting its current property switches screens.

ScreenManager: പേരുള്ള പല Screen object-കൾ ഉൾക്കൊള്ളുകയും ഒരു സമയം ഒന്ന് മാത്രം കാണിക്കുകയും ചെയ്യുന്ന Kivy widget; അതിന്റെ current property മാറ്റിയാൽ screen മാറും.

Each screen is a Screen subclass with a name. In KV, `<ClassName>`: defines a rule for a class, and inside a screen `root.manager.current = "tasks"` switches screens. `Builder.load_string(kv)` loads KV text from Python. Styling uses properties such as padding, spacing, `font_size: "24sp"` and `color: 1, 0, 0, 1` (RGBA, here red). Inside KV, `app` refers to the running App object, so `on_press: app.add_task(task_input.text)` calls a Python method.

മലയാളത്തിൽ

ScreenManager ഒരു TV remote പോലെ: പല channel-കൾ (screen-കൾ) ഉണ്ട്, current മാറ്റിയാൽ വേറെ channel കാണാം. Data save ചെയ്യാൻ `add_task`-ൽ json file-ലേക്ക് എഴുതുന്നു, `app` തുടങ്ങുമ്പോൾ `on_start()`-ൽ അത് തിരികെ വായിക്കുന്നു.

EXAMPLE (THE UNIT'S TWO-SCREEN APP WITH TASK SAVING)

```

import json
from kivy.app import App
from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.lang import Builder

kv = """
ScreenManager:
    MenuScreen:
    TaskScreen:

<MenuScreen>:
    name: "menu"
    BoxLayout:
        orientation: "vertical"
        Label:
            text: "Welcome to Kivy App"
        Button:
            text: "Go to Tasks"
            on_press: root.manager.current = "tasks"

<TaskScreen>:
    name: "tasks"
    BoxLayout:
        orientation: "vertical"
        padding: 20
        spacing: 10
        Label:
            text: "Task Manager"
            font_size: "24sp"
            color: 1, 0, 0, 1
        TextInput:
            id: task_input
            hint_text: "Enter a new task"
        Button:
            text: "Add Task"
            on_press: app.add_task(task_input.text)
        Button:
            text: "Back"
            on_press: root.manager.current = "menu"
"""

class MenuScreen(Screen):
    pass

class TaskScreen(Screen):
    pass

class MyApp(App):
    tasks = []
    data_file = "tasks.json"

```

```

def build(self):
    return Builder.load_string(kv)

def add_task(self, task):
    if task.strip():
        self.tasks.append(task.strip())
        self.save_tasks()
        print("Tasks:", self.tasks)

def save_tasks(self):
    with open(self.data_file, "w") as f:
        json.dump(self.tasks, f)

def on_start(self):
    try:
        with open(self.data_file) as f:
            self.tasks = json.load(f)
    except FileNotFoundError:
        self.tasks = []

if __name__ == "__main__":
    MyApp().run()

```

First screen: "Welcome to Kivy App" with a "Go to Tasks" button. Second screen: a red "Task Manager" heading, a text box with hint "Enter a new task", "Add Task" and "Back". Adding "Buy groceries" prints Tasks: ['Buy groceries'] and writes ["Buy groceries"] to tasks.json; next time the app starts, on_start() reloads the list.

EXAMPLE: RESPONSIVE PLACEMENT (KV)

```

<TaskScreen>:
    RelativeLayout:
        Button:
            text: "Bottom Right"
            size_hint: 0.3, 0.2
            pos_hint: {"right": 1, "y": 0}

```

The button is always 30 % of the width and 20 % of the height, fixed to the bottom-right corner however the window is resized.

Advantages of Kivy	Limitations of Kivy
Write once, run on desktop and mobile	Larger app size, especially on mobile
Open source, actively maintained	Not a natural fit for traditional desktop UIs
Multi-touch and gestures built in	Fewer native-looking widgets
Hardware-accelerated (OpenGL) rendering	

Exam tip: In the unit, the styled <TaskScreen> rule and the one using `app.add_task` are printed separately from the first KV string; to run them together they must be one KV rule per class, as combined above. The KV rule header is written with angle brackets, <TaskScreen>:.

Comparison of Python GUI frameworks

Choose by project need: Tkinter for small tools and learning, PyQt for professional feature-rich desktop apps, wxPython for native-looking desktop apps, PySimpleGUI for the quickest simple forms, Kivy for touch and mobile apps.

മലയാളത്തിൽ

വീട്ടിലെ ചെറിയ കണക്കുകൂട്ടൽ tool-ന് Tkinter മതി; ഒരു company-യുടെ വലിയ desktop software-ന് PyQt; OS-ന്റെ തനതായ look വേണമെങ്കിൽ wxPython; പത്ത് വരിയിൽ ഒരു form വേണമെങ്കിൽ PySimpleGUI; phone-ലും ഓടണമെങ്കിൽ Kivy.

Feature	Tkinter	PyQt	wxPython	PySimpleGUI	Kivy
Included with Python	Yes	No (pip)	No (pip)	No (pip)	No (pip)
Based on	Tcl/Tk	Qt (C++)	wxWidgets (C++)	Tkinter / Qt / wx / Remi	OpenGL (own widgets)
Look	Basic	Modern, native-like, stylable	Native	Depends on back end; themes	Custom, touch-friendly
Cross-platform	Win/Mac/Linux	Win/Mac/Linux	Win/Mac/Linux	Win/Mac/Linux	Win/Mac/Linux + Android/iOS
Event handling	<code>command=</code> , <code>bind()</code>	Signals and slots	<code>Bind(wx.EVT_...)</code>	Own while loop with <code>read()</code>	<code>bind(on_press=...)</code> , KV
Layout	<code>pack</code> , <code>grid</code> , <code>place</code>	<code>QVBoxLayout</code> / <code>QHBoxLayout</code> / <code>QGridLayout</code>	Sizers	List of lists (rows)	<code>Box</code> / <code>Grid</code> / <code>Float</code> / <code>AnchorLayout</code>
Event loop	<code>mainloop()</code>	<code>app.exec()</code>	<code>MainLoop()</code>	<code>window.read()</code> in a loop	<code>run()</code>
Learning curve	Easy	Steeper	Moderate	Easiest	Moderate
Best use	Simple apps, teaching, tools	Professional desktop apps	Native desktop apps	Quick prototypes, small tools	Mobile + desktop, multi-touch

Exam tip: The unit's comparison table lists wxPython's use case as "Native Windows/Linux apps"; wxPython is also native on macOS (the unit itself says so in its feature list). The unit's table also leaves out PySimpleGUI; the extended table above includes it.

Quick recap

- A GUI program builds widgets, arranges them, connects events to callbacks, then runs an event loop.
- Tkinter is built into Python (Tcl/Tk); skeleton: `import tkinter as tk, tk.Tk(), widgets, mainloop()`.

- Key Tkinter widgets: Label, Button, Entry, Text, Checkbutton, Radiobutton, Listbox, Scale, Spinbox, Canvas, Frame, Message, Menu; `config()` changes options at run time.
- Layout managers: pack (stacking), grid (rows/columns), place (exact x/y). Do not mix pack and grid in one container.
- Button `command=func` (no brackets); `bind("<Event>", handler)` handlers take an event argument.
- ttk adds Treeview (tables) and Notebook (tabs); OOP design subclasses `tk.Tk`; `sqlite3` gives persistent storage.
- Long tasks go in a background thread; only the main thread should update Tkinter widgets (`queue + after()`).
- PyQt = Python bindings for Qt: `QApplication`, `QWidget`, `show()`, `app.exec()`; widgets start with `Q`.
- PyQt events use signals and slots: `button.clicked.connect(func)`; layouts `QVBoxLayout`, `QHBoxLayout`, `QGridLayout`; Qt Designer + `pyuic5/pyuic6`.
- wxPython wraps wxWidgets for a native look: `wx.App`, `wx.Frame`, `wx.Panel`, `Bind(wx.EVT_BUTTON, ...)`, sizers such as `wx.BoxSizer`.
- PySimpleGUI: layout is a list of lists; you write the loop event, `values = window.read()`; use `FreeSimpleGUI` as the free fork.
- Kivy: multi-touch, GPU-drawn, runs on mobile; subclass `App`, return the root widget from `build()`, call `run()`; KV language separates design from logic.
- Choose: Tkinter (simple), PyQt (professional), wxPython (native), PySimpleGUI (fastest to write), Kivy (touch/mobile).

Check yourself

Q. 1. Which module do you import to use Tkinter in Python 3, and which method starts its event loop?

tkinter (lowercase), usually as `import tkinter as tk`. The event loop is started with `root.mainloop()`.

Q. 2. Fill in the blanks: (a) the ____ widget takes single-line text input; (b) the ____ widget draws lines and circles; (c) the ____ method changes a label's text at run time.

(a) Entry (b) Canvas (c) `config()` (or `configure()`).

Q. 3. Write a short snippet to create a window with the title "Welcome".

```
import tkinter as tk
root = tk.Tk()
root.title("Welcome")
root.mainloop()
```

An empty window with "Welcome" in its title bar.

Q. 4. Write a Tkinter program with an Entry box and a button; when the button is pressed the typed text appears in a Label.

```
import tkinter as tk

def display_input():
    label.config(text="You entered: " + entry.get())

root = tk.Tk()
root.title("Check Your Progress")
entry = tk.Entry(root, width=25)
entry.pack(pady=10)
tk.Button(root, text="Submit", command=display_input).pack(pady=5)
label = tk.Label(root, text="", font=("Arial", 14))
label.pack(pady=10)
root.mainloop()
```

Typing "Hello" and clicking Submit shows "You entered: Hello" below the button.

Q. 5. In a Tkinter student manager with a ttk.Treeview, how should the "Delete Student" button remove the selected row from both the table and the Python list students?

```
def delete_student():
    selected = tree.selection()
    if selected:
        idx = tree.index(selected[0])
        students.pop(idx)
        tree.delete(selected[0])
    else:
        messagebox.showwarning("Delete Error", "No student selected!")
```

Use the row's position (`tree.index()`) to remove it from the list. The unit's answer uses `students.remove(tree.item(selected) ['values'] )`, which fails: Treeview returns the values as a list and converts numeric strings such as "12" to the integer 12, so it never equals the stored tuple of strings ("Rohan", "12", "85"), and `remove()` raises `ValueError`.

Q. 6. What is the purpose of QApplication, and which method makes a PyQt window visible?

`QApplication` initialises and manages the GUI application and runs its event loop (`app.exec()`); one is needed per program. A window is made visible with `show()`.

Q. 7. Explain signals and slots in PyQt with a short example.

A signal is emitted by a widget when an event happens (e.g. clicked); a slot is a function connected to it that runs whenever it is emitted.

```
def print_message():  
    print("Button was clicked")  
  
btn.clicked.connect(print_message)
```

Note the function must be defined before `connect()` is executed; the unit's answer writes the `connect` line first, which would give a `NameError` at module level.

Q. 8. What is a layout manager in PyQt? Name two. How is a wxPython sizer similar?

A layout manager arranges widgets in a window and automatically handles their position and size when the window is resized. Examples: `QVBoxLayout` (vertical stack) and `QGridLayout` (rows and columns); also `QHBoxLayout`. A wxPython sizer (e.g. `wx.BoxSizer`) does the same job in wxPython.

Q. 9. Which class creates the main window in wxPython, and what does `wx.App` do? Differentiate `wx.TextCtrl` and `wx.StaticText`.

`wx.Frame` is the main window. `wx.App` initialises wxPython and runs the main loop (`MainLoop()`). `wx.TextCtrl` is an editable text field for user input; `wx.StaticText` displays read-only text.

Q. 10. What is the purpose of the event loop in PySimpleGUI? How is it different from Tkinter?

The loop `while True: event, values = window.read()` repeatedly waits for user events and returns the event name and all input values, so the program can react; it ends when the window is closed, then `window.close()` is called. In Tkinter, the loop is hidden inside `mainloop()` and you attach callbacks; in PySimpleGUI you write the loop yourself and test the event with `if` statements.

Q. 11. What is Kivy, what is the role of the KV language, and what do `build()` and `run()` do?

Kivy is an open-source Python framework for cross-platform (desktop and mobile), multi-touch, GPU-accelerated apps. The KV language is a declarative language that describes the UI separately from the Python logic, making code cleaner and easier to maintain. `build()` returns the root widget of the app; `run()` starts the main loop and displays the interface.

Q. 12. Which framework would you choose for (a) a quick internal tool with no installation, (b) a professional desktop application with complex tables, (c) an app that must also run on Android? Justify.

(a) Tkinter: ships with Python, easy and lightweight. (b) PyQt: rich widgets (QTableView, model-view), Qt Designer and styling. (c) Kivy: the same code runs on Android and iOS with touch support.

UNIT 10

Creating RESTful APIs in Python

രണ്ട് software-കൾ internet വഴി പരസ്പരം സംസാരിക്കുന്നത് API വഴിയാണ്. ഈ unit-ൽ REST, HTTP methods, status codes, JSON, CRUD എന്നിവ മനസ്സിലാക്കി, Flask-ഉം FastAPI-ഉം ഉപയോഗിച്ച് സ്വന്തമായി RESTful API ഉണ്ടാക്കി test ചെയ്യാൻ പഠിക്കുന്നു.

What is an API?

Web APIs, HTTP and the Google Translate API

REST and its constraints

Structure of an API request

Structure of an API response and status codes

JSON: the data format of REST APIs

CRUD operations and HTTP methods

Installing Flask, FastAPI and other tools

A first Flask app and returning JSON

CRUD endpoints in Flask: the Book API

Calling an API: requests and curl

Input validation and error handlers in Flask

Advanced Flask: SQLite, token auth and pagination

What is FastAPI? Your first FastAPI app

FastAPI routes with Pydantic models

Protecting a FastAPI endpoint

Advanced FastAPI: async, dependencies and background tasks

Flask vs FastAPI

What is an API?

API (Application Programming Interface): a set of rules that allows two software applications to communicate and exchange data with each other.

API (Application Programming Interface): രണ്ട് software applications-ന് പരസ്പരം communicate ചെയ്യാനും data കൈമാറാനും അനുവദിക്കുന്ന ഒരു കൂട്ടം നിയമങ്ങൾ.

An API acts as a bridge or translator between two programs. Neither program needs to know how the other works inside; it only needs to know what requests it can send and what responses it will get back. When a cab-booking app shows nearby cars, driver location and fare, that data comes from the company's server through an API.

മലയാളത്തിൽ

ഒരു ഹോട്ടലിൽ ചെന്നാൽ നമ്മൾ നേരെ അടുക്കളയിൽ കയറി ചോറും മീൻകറിയും എടുക്കാറില്ല. Waiter-നോട് order പറയും, waiter അത് അടുക്കളയിൽ (server) എത്തിക്കും, ഭക്ഷണം (response) തിരികെ കൊണ്ടുവരും. ഈ waiter ആണ് API. അടുക്കളയിൽ എന്ത് നടക്കുന്നു എന്ന് customer അറിയേണ്ടതില്ല; menu-വിൽ ഉള്ളത് ചോദിച്ചാൽ മതി.

Customer = client (app) → Waiter = API → Kitchen = server
Order = request ← Food = response

EXAMPLE

A weather app asks an API "What is the weather in Delhi today?". What does a typical response look like?
The server returns structured data (JSON), which the app then displays:

```
{"city": "Delhi", "temperature": "32°C", "status": "Sunny"}
```

The app never touches the weather company's database. It only sends a request and reads the response.

APIs let developers (i) use features or data of another service (Google Maps, payment gateways), (ii) request or send data without connecting directly to the service's database, and (iii) build apps faster with ready-made services. APIs are not only for the web: operating systems, games and desktop software also expose APIs.

Exam tip: For "Explain API with a real-world example", write the definition, then the restaurant analogy (customer, waiter, kitchen) or the cab-booking app, and end with one line: the client never accesses the server's database directly.

Web APIs, HTTP and the Google Translate API

Web API: an API that is accessed over the web using standard web protocols, most commonly HTTP.

HTTP (Hypertext Transfer Protocol): the request-response protocol that browsers and apps use to exchange data with web servers.

Web API: HTTP പോലുള്ള standard web protocols ഉപയോഗിച്ച് web വഴി access ചെയ്യുന്ന API.

HTTP (Hypertext Transfer Protocol): browsers-ഉം apps-ഉം web servers-മായി data കൈമാറാൻ ഉപയോഗിക്കുന്ന request-response protocol.

A client (browser, mobile app, Python script) sends an **HTTP request** to a server; the server sends back an **HTTP response** containing a status code and usually data in JSON (sometimes XML). Well-known Web APIs include Google Maps, Twitter (X), weather APIs and Google Translate.

മലയാളത്തിൽ

Browser web page തുറക്കാൻ ഉപയോഗിക്കുന്ന അതേ HTTP തന്നെയാണ് Web API-യും ഉപയോഗിക്കുന്നത്. വ്യത്യാസം ഒന്നു മാത്രം: web page HTML തിരികെ തരുമ്പോൾ Web API സാധാരണ JSON data ആണ് തരുന്നത്, അത് program-ന് നേരിട്ട് വായിക്കാം.

EXAMPLE: GOOGLE TRANSLATE API

You must translate 2098 text files from English to Spanish. Copy-pasting each into the Google Translate web page is tedious. Instead, a program sends one HTTP POST request per file to the Cloud Translation REST API:

```
POST https://translation.googleapis.com/language/translate/v2?key=YOUR_API_KEY
Content-Type: application/json
```

```
{
  "q": "Hello world",
  "source": "en",
  "target": "es",
  "format": "text"
}
```

The response is JSON of the form `{"data": {"translations": [{"translatedText": "Hola Mundo"}]}}`. A loop over the files makes the job automatic. (The API key comes from a Google Cloud Console project with the Cloud Translation API enabled. Real calls need the key and network access, so this one is not run here.)

REST and its constraints

REST (Representational State Transfer): an architectural style for designing networked applications, introduced by Roy Fielding in 2000, where resources identified by URLs are handled with standard HTTP methods.

RESTful API: a Web API that follows the REST constraints.

REST (Representational State Transfer): 2000-ൽ Roy Fielding അവതരിപ്പിച്ച, networked applications design ചെയ്യാനുള്ള ഒരു architectural style. ഇതിൽ URL കൊണ്ട് തിരിച്ചറിയുന്ന resources-നെ standard HTTP methods ഉപയോഗിച്ച് കൈകാര്യം ചെയ്യുന്നു.

RESTful API: REST constraints പാലിക്കുന്ന Web API.

REST is not a standard or a library; it is a set of guiding principles that make web services simple, scalable and stateless. IGNOU lists six constraints a RESTful API follows:

Constraint	Meaning
Statelessness	Every request carries all the information needed to process it; the server stores no client session.
Client-Server	Client (app) and server (data) are separate, so each can be developed independently.
Uniform Interface	All resources are handled in one consistent way, using standard HTTP methods.
Resource-Based	Everything (users, products, orders) is a resource with its own URL, e.g. /products/5.
Representation of Resources	Clients receive representations of resources (JSON or XML), not the raw stored data.
Cacheable	Responses state whether they can be cached, which improves performance.

മലയാളത്തിൽ

Stateless എന്നാൽ server-ന് "ഓർമ്മ" ഇല്ല എന്നർത്ഥം. KSRTC ticket counter-ൽ ഓരോ തവണയും നിങ്ങൾ എവിടെ നിന്ന് എവിടേക്ക്, എത്ര പേർ എന്ന് മുഴുവൻ പറയണം; കഴിഞ്ഞ തവണ പറഞ്ഞത് clerk ഓർത്തുവെക്കില്ല. അതുകൊണ്ട് ഏത് counter-ൽ പോയാലും കാര്യം നടക്കും. അതുപോലെ stateless API-യിൽ ഏത് server-ന് വേണമെങ്കിലും request കൈകാര്യം ചെയ്യാം, അതിനാൽ scale ചെയ്യാൻ എളുപ്പമാണ്.

EXAMPLE: WHY STATELESSNESS MATTERS

A railway booking API runs on 3 servers behind a load balancer. What goes wrong if it is stateful?

If server A remembers "user selected train 12626, seat 34" in its memory and the next request is sent to server B, B knows nothing and the booking breaks. The server also needs memory for every logged-in user, and one crash loses every session. In a stateless design each request carries everything needed (for example a token plus the train and seat), so any server can answer it and servers can be added freely.

Exam tip: Write IGNOU's six in the exam. Know that Fielding's original list is slightly different: client-server, stateless, cacheable, uniform interface, layered system and code on demand (optional). IGNOU's "resource-based" and "representation" are really parts of the uniform interface constraint.

Structure of an API request

HTTP request: a message from client to server made of an HTTP method, a URL identifying the resource, optional headers and an optional body.

Endpoint: a URL (with its method) at which an API exposes a resource, e.g. GET /products/5.

HTTP request: client server-ലേക്ക് അയക്കുന്ന message. അതിൽ ഒരു HTTP method, resource തിരിച്ചറിയുന്ന URL, ആവശ്യമെങ്കിൽ headers, body എന്നിവ ഉണ്ടാകും.

Endpoint: API ഒരു resource ലഭ്യമാക്കുന്ന URL (അതിന്റെ method-ഉം ചേർത്ത്), ഉദാ. GET /products/5.

(a) **Method** says what action is wanted. (b) **URL** says which resource. (c) **Headers** are metadata, e.g. Content-Type: application/json (the body is JSON), Authorization: Bearer <token> (security token), Accept: application/json (the client wants JSON back). (d) **Body** carries data and is used with POST, PUT and PATCH.

മലയാളത്തിൽ

Post office-ൽ ഒരു registered letter അയക്കുന്നത് ഓർക്കുക. Method = എന്ത് സേവനം വേണം (അയക്കണോ, തിരികെ വാങ്ങണോ), URL = വിലാസം, headers = കവറിന് പുറത്തെ വിവരങ്ങൾ (ഭാഷ, അയച്ച ആളുടെ ID), body = കവറിനകത്തെ കത്ത്. GET request-ന് സാധാരണ body ഉണ്ടാകില്ല.

Method	Action	Example	Has body?
GET	Read / fetch data	GET /products	No
POST	Create new data	POST /users	Yes
PUT	Update a resource entirely (replace)	PUT /products/5	Yes
PATCH	Update part of a resource	PATCH /products/5 (only price)	Yes
DELETE	Remove a resource	DELETE /comments/9	Usually no

EXAMPLE

Write the full request that creates a user Alice.

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer abc123

{"name": "Alice", "email": "alice@example.com"}
```

Method = POST, URL = /users, headers = Content-Type and Authorization, body = the JSON object.

EXAMPLE: GET VS POST

GET	POST
Retrieves data	Sends data, usually to create something
Data goes in the URL as query parameters (/search?name=python)	Data goes in the request body
Visible in the address bar, history and logs	Not visible in the URL
Safe and idempotent: repeating it changes nothing	Not idempotent: repeating it can create duplicates

Exam tip: The unit says POST data is "hidden". It is only kept out of the URL; the body is still plain text unless HTTPS is used. Also remember that PUT and DELETE are idempotent too: sending the same PUT twice leaves the same result.

Structure of an API response and status codes

HTTP response: the server's reply, made of a status code, response headers and a response body (usually JSON).

Status code: a three-digit number in the response that tells the client whether the request succeeded or why it failed.

HTTP response: server-ന്റെ മറുപടി. അതിൽ status code, response headers, response body (സാധാരണ JSON) എന്നിവ ഉണ്ടാകും.

Status code: request വിജയിച്ചോ, ഇല്ലെങ്കിൽ എന്തുകൊണ്ട് പരാജയപ്പെട്ടു എന്ന് client-നെ അറിയിക്കുന്ന മൂന്നാക്ക സംഖ്യ.

The first digit gives the class: 2xx success, 3xx redirection, 4xx client error (the request was wrong), 5xx server error (the server failed). Common response headers are Content-Type: application/json, Cache-Control (how long to cache) and Set-Cookie.

മലയാളത്തിൽ

ബാങ്കിൽ cheque കൊടുത്താൽ "പണം ക്രെഡിറ്റ് ആയി" (2xx), "ഒപ്പ് ശരിയല്ല, നിങ്ങളുടെ തെറ്റ്" (4xx), "server down ആണ്, ഞങ്ങളുടെ തെറ്റ്" (5xx) എന്നിങ്ങനെ മറുപടി കിട്ടും. Status code-ന്റെ ആദ്യ അക്കം നോക്കിയാൽ തെറ്റ് ആരുടേതാണെന്ന് മനസ്സിലാക്കും.

Code	Name	When it is used
200	OK	Request succeeded (GET, PUT, DELETE)
201	Created	A new resource was created (POST)
204	No Content	Success with an empty body (often DELETE)
400	Bad Request	Invalid or missing data in the request
401	Unauthorized	No or invalid credentials / token
403	Forbidden	Logged in, but not allowed to do this
404	Not Found	Resource or URL does not exist
405	Method Not Allowed	URL exists but not for this method
415	Unsupported Media Type	Body is not in the expected format (e.g. not JSON)
422	Unprocessable Entity	Body is well-formed but fails validation (FastAPI uses this)
500	Internal Server Error	An error inside the server code

EXAMPLE

Response bodies for reading and for deleting a product:

```
{"id": 5, "product_name": "Wireless Mouse", "price": 499}

{"message": "Product deleted successfully"}
```

A real response captured from the Flask Book API of topic 10 with `curl -i` shows all three parts: status line, headers, body.

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Wed, 30 Sep 2026 11:09:33 GMT
Content-Type: application/json
Content-Length: 33
Connection: close

{"id":1,"title":"Python Basics"}
```

Exam tip: 401 vs 403 is a favourite: 401 = "who are you?" (not authenticated), 403 = "I know you, but you may not" (not authorised).

JSON: the data format of REST APIs

JSON (JavaScript Object Notation): a lightweight, human-readable text format that represents data as key-value pairs (objects) and ordered lists (arrays).

JSON (JavaScript Object Notation): data-യെ key-value pairs (objects) ആയും ക്രമമുള്ള lists (arrays) ആയും കാണിക്കുന്ന, ലളിതവും മനുഷ്യസ്വാധീനവാചകവുമായ ഒരു text format.

REST APIs use JSON for both input (request body) and output (response body) because it is small and easy to parse in every language. Python's built-in `json` module converts between the two: `json.dumps()` turns a Python object into a JSON string, and `json.loads()` turns a JSON string into a Python object.

മലയാളത്തിൽ

JSON കാണാൻ Python dictionary പോലെ തന്നെ. പക്ഷേ ചെറിയ വ്യത്യാസങ്ങൾ ഉണ്ട്: JSON-ൽ strings-ന് double quotes മാത്രം, True-ക്ക് പകരം `true`, None-ക്ക് പകരം `null`. Python-ഉം JSON-ഉം തമ്മിൽ മാറ്റാൻ `dumps`, `loads` ഉപയോഗിക്കുന്നു.

`json.dumps(obj)` : Python object → JSON string
`json.loads(s)` : JSON string → Python object
dict ↔ object, list ↔ array, True/False ↔ true/false, None ↔ null

EXAMPLE

```
import json

user = {"name": "Rahul", "email": "rahul@example.com", "isActive": True}
text = json.dumps(user)
print(text)
print(type(text))

back = json.loads('{"city": "Delhi", "temperature": 32, "status": "Sunny", "rain": null}')
print(back)
print(back["city"], type(back["temperature"]))
```

```
{"name": "Rahul", "email": "rahul@example.com", "isActive": true}
<class 'str'>
{'city': 'Delhi', 'temperature': 32, 'status': 'Sunny', 'rain': None}
Delhi <class 'int'>
```

True became true in JSON, and null became None in Python. In Flask, `jsonify()` and `request.get_json()` do this conversion for you.

CRUD operations and HTTP methods

CRUD: the four basic data operations Create, Read, Update and Delete, which REST APIs map to the HTTP methods POST, GET, PUT/PATCH and DELETE.

CRUD: Create, Read, Update, Delete എന്ന നാല് അടിസ്ഥാന data operations. REST API-യിൽ ഇവ യഥാക്രമം POST, GET, PUT/PATCH, DELETE എന്നീ HTTP methods-മായി ബന്ധിപ്പിക്കുന്നു.

A resource usually has two URL patterns: the **collection** (`/students`) and a **single item** (`/students/{id}`). The method, not the URL, decides the action, so URLs are nouns (`/students`), never verbs (`/getStudents`).

മലയാളത്തിൽ

College office-ലെ student register ഓർക്കുക: പുതിയ admission ചേർക്കൽ (Create), പേര് നോക്കൽ (Read), course മാറ്റൽ (Update), TC വാങ്ങി പോയവരെ നീക്കൽ (Delete). Register ഒന്നു തന്നെ (URL), ചെയ്യുന്ന പ്രവൃത്തി മാത്രം മാറുന്നു (method).

CRUD	HTTP method	Endpoint	Description	Success code
Create	POST	/students	Add a new student	201
Read (all)	GET	/students	List all students	200
Read (one)	GET	/students/{id}	Details of one student	200
Update	PUT (full) / PATCH (partial)	/students/{id}	Change a student's data	200
Delete	DELETE	/students/{id}	Remove a student	200 or 204

EXAMPLE: RAILWAY RESERVATION API

Design endpoints for a railway reservation system.

Resources: trains, stations, bookings, passengers, payments.

GET	/trains	list trains (?from=ERS&to=TVC&date=2026-10-02)
GET	/trains/{id}	details of one train
POST	/bookings	create a booking -> 201
GET	/bookings/{pnr}	booking status (PNR) -> 200 / 404
PATCH	/bookings/{pnr}	change boarding point
DELETE	/bookings/{pnr}	cancel the booking -> 200

Request for POST /bookings: {"train_id": 16301, "date": "2026-10-02", "passengers": [{"name": "Anu", "age": 24}]} Response (201): {"pnr": "4521367890", "status": "CNF", "coach": "S4", "seat": 34}.

Installing Flask, FastAPI and other tools

pip: Python's package manager, used to install libraries from PyPI.

Uvicorn: an ASGI (Asynchronous Server Gateway Interface) web server used to run FastAPI applications.

pip: PyPI-ൽ നിന്ന് libraries install ചെയ്യാനുള്ള Python-ന്റെ package manager.

Uvicorn: FastAPI applications run ചെയ്യാൻ ഉപയോഗിക്കുന്ന ASGI (Asynchronous Server Gateway Interface) web server.

Check Python and pip, then install the frameworks. `requests` lets Python code call APIs; `httpie` is a friendly command-line alternative to `curl`.

മലയാളത്തിൽ

ആദ്യം Python-ഉം pip-ഉം ഉണ്ടോ എന്ന് നോക്കുക, പിന്നെ ആവശ്യമുള്ള libraries install ചെയ്യുക. FastAPI എന്നത് framework മാത്രം; അതിനെ run ചെയ്യാൻ Uvicorn എന്ന server കൂടി വേണം. Flask-ന് development server അതിനകത്ത് തന്നെയുണ്ട്.

```
python --version      # or: python3 --version
pip --version         # or: pip3 --version
pip install Flask
python -m flask --version
pip install fastapi "uvicorn[standard]"
pip install requests httpie
```

Exam tip: The unit prints the commands as `python--version` and `pip -- version` (a typesetting error). The correct commands use two plain hyphens and no space: `python --version`, `pip --version`.

A first Flask app and returning JSON

Flask: a lightweight, minimal Python web framework for building web applications and APIs.

Route: a mapping, made with `@app.route()`, from a URL (and method) to the Python function that handles it.

Flask: web applications-ഉം APIs-ഉം നിർമ്മിക്കാനുള്ള ലളിതവും ഭാരം കുറഞ്ഞതുമായ ഒരു Python web framework.

Route: ഒരു URL-നെ (method-ഉം ചേർത്ത്) അത് കൈകാര്യം ചെയ്യുന്ന Python function-മായി ബന്ധിപ്പിക്കുന്നത്; `@app.route()` ഉപയോഗിച്ചാണ് ഉണ്ടാക്കുന്നത്.

`Flask(__name__)` creates the app. Each `@app.route(path, methods=[...])` decorator attaches a function (the view function) to a URL. Whatever the function returns becomes the response. `jsonify()` converts a Python list or dict to a JSON response with `Content-Type: application/json`. `app.run(debug=True)` starts a server at `http://127.0.0.1:5000/`.

മലയാളത്തിൽ

Route എന്നത് ഒരു വീട്ടിലെ calling bell പോലെയാണ്: ഓരോ bell-ഉം (URL) ഓരോ മുറിയിലെ ആളെ (function) വിളിക്കുന്നു. `/books` എന്ന URL-ൽ GET വന്നാൽ `get_books()` run ആകും. Function തിരിച്ചു തരുന്നത് client-ന് കിട്ടുന്ന response.

EXAMPLE 1: BASIC APP

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to the Book API!"

if __name__ == '__main__':
    app.run(debug=True)
```

Visiting `http://127.0.0.1:5000/` (tested with `app.test_client().get('/')`):

```
200 OK Welcome to the Book API!
```

EXAMPLE 2: RETURNING JSON

```
from flask import jsonify

@app.route('/books', methods=['GET'])
def get_books():
    books = [
        {"id": 1, "title": "Python Basics"},
        {"id": 2, "title": "Flask for Beginners"}
    ]
    return jsonify(books)
```

```
200 OK [{"id":1,"title":"Python Basics"},{"id":2,"title":"Flask for Beginners"}]
```

EXAMPLE 3: TWO ENDPOINTS AND A QUERY PARAMETER (CHECK YOUR PROGRESS 2)

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route('/hello', methods=['GET'])
def say_hello():
    return "Hello, World!"

@app.route('/goodbye', methods=['GET'])
def say_goodbye():
    return "Goodbye!"

@app.route('/square', methods=['GET'])
def calculate_square():
    num = request.args.get('num', default=0, type=int)
    return jsonify({"result": num * num})
```

Responses for /hello, /goodbye, /square?num=5, /square, /square?num=abc:

```
200 OK Hello, World!
200 OK Goodbye!
200 OK {"result":25}
200 OK {"result":0}
200 OK {"result":0}
```

`request.args` holds the query parameters (the part after `?`). With `type=int`, a missing or non-numeric value silently falls back to the default 0.

Exam tip: The unit's code has errors you must not copy: `if __name__ == '__main__':` uses curly quotes (a `SyntaxError`; use straight quotes), and the `/square` answer has `@app.route('/square, methods=['GET'])` with a missing closing quote. The unit also says `home()` shows "Welcome to the Flask API!", but the code returns "Welcome to the Book API!".

CRUD endpoints in Flask: the Book API

Path parameter: a variable part of the URL, written `<int:book_id>` in Flask, whose value is passed to the view function as an argument.

`request.get_json()`: the Flask call that reads the JSON body sent by the client and returns it as a Python dict or list.

Path parameter: URL-ലെ മാറ്റുന്ന ഭാഗം; Flask-ൽ `<int:book_id>` എന്ന് എഴുതുന്നു. അതിന്റെ value view function-ന് argument ആയി കിട്ടും.

`request.get_json()`: client അയച്ച JSON body വായിച്ച് Python dict അല്ലെങ്കിൽ list ആയി തരുന്ന Flask call.

The full Book API keeps books in a Python list and exposes all CRUD operations. A view can return a tuple (response, status_code), e.g. `jsonify(...), 201`. If no book matches, the API returns 404 with an error message.

മലയാളത്തിൽ

ഒരേ URL `/books/<id>`-ന് GET, PUT, DELETE എന്നിങ്ങനെ മൂന്ന് functions ഉണ്ട്; ഏത് method-ൽ request വന്നു എന്ന് നോക്കി Flask ശരിയായ function തിരഞ്ഞെടുക്കും. `<int:book_id>` എന്ന് എഴുതിയതുകൊണ്ട് URL-ലെ "3" എന്നത് integer 3 ആയി function-ന് കിട്ടും.

EXAMPLE: COMPLETE BOOK API

```

from flask import Flask, request, jsonify

app = Flask(__name__)

books = [
    {"id": 1, "title": "Python Basics"},
    {"id": 2, "title": "Flask for Beginners"}
]

@app.route('/')
def home():
    return "Welcome to the Book API!"

@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

@app.route('/books', methods=['POST'])
def add_book():
    new_book = request.get_json()
    books.append(new_book)
    return jsonify({"message": "Book added successfully"}), 201

@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    for book in books:
        if book['id'] == book_id:
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404

@app.route('/books/<int:book_id>', methods=['PUT'])
def update_book(book_id):
    updated_data = request.get_json()
    for book in books:
        if book['id'] == book_id:
            book.update(updated_data)
            return jsonify(book)
    return jsonify({"error": "Book not found"}), 404

@app.route('/books/<int:book_id>', methods=['DELETE'])
def delete_book(book_id):
    for book in books:
        if book['id'] == book_id:
            books.remove(book)
            return jsonify({"message": "Book deleted"})
    return jsonify({"error": "Book not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Testing without a browser using Flask's built-in test client:

```

c = app.test_client()
def show(r):
    print(r.status, r.get_data(as_text=True).strip())

show(c.get('/books'))
show(c.post('/books', json={"id": 3, "title": "API Development in Python"}))
show(c.get('/books/3'))
show(c.put('/books/3', json={"title": "Advanced API Development"}))
show(c.delete('/books/3'))
show(c.get('/books/3'))

```

```

200 OK [{"id":1,"title":"Python Basics"}, {"id":2,"title":"Flask for Beginners"}]
201 CREATED {"message":"Book added successfully"}
200 OK {"id":3,"title":"API Development in Python"}
200 OK {"id":3,"title":"Advanced API Development"}
200 OK {"message":"Book deleted"}
404 NOT FOUND {"error":"Book not found"}

```

Two more cases: DELETE /books (no such route for that method) gives **405 METHOD NOT ALLOWED**, and GET /books/abc gives **404** because <int:...> does not match "abc".

EXAMPLE: SEARCH BY KEYWORD (CHECK YOUR PROGRESS 2, Q5)

```

@app.route('/books/search', methods=['GET'])
def search_books():
    keyword = request.args.get('keyword', '').lower()
    results = [book for book in books if keyword in book['title'].lower()]
    if results:
        return jsonify(results)
    return jsonify({"message": "No books found with given keyword"}), 404

```

GET /books/search?keyword=flask and ?keyword=java:

```

200 OK [{"id":2,"title":"Flask for Beginners"}]
404 NOT FOUND {"message":"No books found with given keyword"}

```

Use GET (it only reads) with the keyword as a query parameter.

Exam tip: The unit says "PUT replaces the existing book data", but `book.update(updated_data)` merges: the output above kept "id": 3 even though the body sent only the title. Merging is really PATCH behaviour. A true PUT would replace the whole record (e.g. `books[i] = {"id": book_id, **updated_data}`).

Calling an API: requests and curl

requests: a Python library for sending HTTP requests, used to call (consume) APIs from Python code.

curl: a command-line tool for sending HTTP requests, commonly used to test APIs.

requests: HTTP requests അയക്കാനുള്ള Python library; Python code-ൽ നിന്ന് APIs വിളിക്കാൻ ഉപയോഗിക്കുന്നു.

curl: HTTP requests അയക്കാനുള്ള command-line tool; APIs test ചെയ്യാൻ സാധാരണ ഉപയോഗിക്കുന്നു.

So far we wrote the server side. The client side can be a mobile app, a browser, or a Python program using requests: `requests.get/post/put/delete(url, json=...)` sends the request, `r.status_code` gives the code, `r.json()` parses the JSON body, and `r.ok` is True for codes below 400.

മലയാളത്തിൽ

Flask-ൽ ഉണ്ടാക്കിയത് അടുക്കള (server). ഇനി customer-ന്റെ ഭാഗം: requests library ഉപയോഗിച്ച് Python program തന്നെ order കൊടുക്കുന്നു. `json=` എന്ന് കൊടുത്താൽ dictionary JSON ആക്കി Content-Type header-ഉം അത് തന്നെ ചേർക്കും.

EXAMPLE 1: PYTHON CLIENT

With the Book API of topic 10 running on 127.0.0.1:5000:

```
import requests

BASE = "http://127.0.0.1:5000"

r = requests.get(f"{BASE}/books")
print(r.status_code, r.headers["Content-Type"])
print(r.json())

r = requests.post(f"{BASE}/books", json={"id": 3, "title": "Learn REST APIs"})
print(r.status_code, r.json())

r = requests.put(f"{BASE}/books/3", json={"title": "REST APIs in Python"})
print(r.status_code, r.json())

r = requests.delete(f"{BASE}/books/3")
print(r.status_code, r.json())

r = requests.get(f"{BASE}/books/3")
print(r.status_code, r.json(), r.ok)
```

```
200 application/json
[{'id': 1, 'title': 'Python Basics'}, {'id': 2, 'title': 'Flask for Beginners'}]
201 {'message': 'Book added successfully'}
200 {'id': 3, 'title': 'REST APIs in Python'}
200 {'message': 'Book deleted'}
404 {'error': 'Book not found'} False
```

`r.json()` gives a Python dict, which is why the output shows single quotes.

EXAMPLE 2: CURL

```
curl -X POST http://127.0.0.1:5000/books \  
-H "Content-Type: application/json" \  
-d '{"id": 3, "title": "API Development in Python"}' \  
-w ' [%{http_code}]\n'  
  
curl -X DELETE http://127.0.0.1:5000/books/9 -w ' [%{http_code}]\n'
```

```
{"message": "Book added successfully"}  
[201]  
{"error": "Book not found"}  
[404]
```

-X sets the method, -H adds a header, -d sends the body, -w '%{http_code}' prints the status code, and -i (used in topic 5) also prints the status line and headers.

Input validation and error handlers in Flask

Input validation: checking that data sent by the client is present and in the correct form before using it, and rejecting it with 400 if not.

Error handler: a function registered with `@app.errorhandler(code)` that builds the response for that error code across the whole app.

Input validation: client അയച്ച data ഉപയോഗിക്കുന്നതിന് മുമ്പ് അത് ഉണ്ടോ, ശരിയായ രൂപത്തിലാണോ എന്ന് പരിശോധിക്കുകയും, ഇല്ലെങ്കിൽ 400 നൽകി നിരസിക്കുകയും ചെയ്യുന്നത്.

Error handler: `@app.errorhandler(code)` ഉപയോഗിച്ച് register ചെയ്യുന്ന function; മുഴുവൻ app-ലും ആ error code-ന്റെ response ഇത് തയ്യാറാക്കുന്നു.

Never trust client data: a missing or empty title should be rejected with 400, not stored. Global error handlers give every error the same JSON shape and hide internal details such as stack traces from clients, which improves both security and user experience.

മലയാളത്തിൽ

Bank counter-ൽ form കൊടുക്കുമ്പോൾ clerk ആദ്യം പേര്, ഒപ്പ്, തുക എല്ലാം ഉണ്ടോ എന്ന് നോക്കും; ഇല്ലെങ്കിൽ അപ്പോൾ തന്നെ തിരിച്ചു തരും. അതാണ് validation. പിന്നെ system-ൽ എന്ത് തകരാർ വന്നാലും "ഇപ്പോൾ സേവനം ലഭ്യമല്ല" എന്ന ഒരു മാനുവായ മറുപടി മാത്രം പറയും, അകത്തെ പ്രശ്നം customer-നോട് വിശദീകരിക്കില്ല. അതാണ് error handler.

EXAMPLE 1: REJECT A MISSING OR EMPTY TITLE (CHECK YOUR PROGRESS 2, Q7)

```
@app.route('/books', methods=['POST'])
def add_book():
    new_book = request.get_json(silent=True)
    title = new_book.get('title') if isinstance(new_book, dict) else None
    if not isinstance(title, str) or not title.strip():
        return jsonify({"error": "Book title is required"}), 400
    books.append(new_book)
    return jsonify({"message": "Book added successfully"}), 201
```

Bodies sent: {"id": 4}, {"id": 4, "title": " "}, {"id": 5, "title": "Machine Learning with Python"}, and the plain text not json:

```
400 BAD REQUEST {"error":"Book title is required"}
400 BAD REQUEST {"error":"Book title is required"}
201 CREATED {"message":"Book added successfully"}
400 BAD REQUEST {"error":"Book title is required"}
```

EXAMPLE 2: GLOBAL 404 AND 500 HANDLERS (CHECK YOUR PROGRESS 2, Q8)

```
@app.errorhandler(404)
def not_found_error(error):
    return jsonify({"error": "Resource not found"}), 404

@app.errorhandler(500)
def internal_error(error):
    return jsonify({"error": "Internal Server Error. Please try again later."}), 500

@app.route('/crash')
def crash():
    return 1 / 0
```

GET /unknown and GET /crash:

```
404 NOT FOUND {"error":"Resource not found"}
500 INTERNAL SERVER ERROR {"error":"Internal Server Error. Please try again
later."}
```

Without the 404 handler, an undefined endpoint returns Flask's default HTML "404 Not Found" page. The ZeroDivisionError traceback goes only to the server log, never to the client.

Exam tip: The unit's answer uses `request.get_json()` and `new_book['title'].strip()`. In Flask 3, a non-JSON body makes `get_json()` abort with **415 Unsupported Media Type**, and a non-string title such as 123 crashes `.strip()` with a 500. Use `get_json(silent=True)` (returns None instead) and check the type, as above.

Advanced Flask: SQLite, token auth and pagination

Bearer token authentication: the client sends a secret token in the header `Authorization: Bearer <token>`, and the server serves the request only if the token is valid.

Pagination: returning a large result set in pages, controlled by parameters such as `offset` (rows to skip) and `limit` (rows per page).

Bearer token authentication: client `Authorization: Bearer <token>` എന്ന header-ൽ ഒരു രഹസ്യ token അയക്കുന്നു; token ശരിയാണെങ്കിൽ മാത്രം server request സ്വീകരിക്കുന്നു.

Pagination: വലിയ result set-നെ പേജുകളായി തിരിച്ച് നൽകുന്നത്; `offset` (ഒഴിവാക്കേണ്ട rows), `limit` (ഒരു പേജിലെ rows) പോലുള്ള parameters കൊണ്ട് നിയന്ത്രിക്കുന്നു.

Data in a Python list is lost when the server restarts, cannot be queried, and does not scale. The Student Management API stores students in SQLite (students(id INTEGER PRIMARY KEY, name TEXT NOT NULL, course TEXT NOT NULL)). Key pieces: g holds one DB connection per request; @app.teardown_appcontext closes it; row_factory = sqlite3.Row lets dict(row) build JSON; a require_token decorator protects POST, PUT and DELETE; and ? placeholders prevent SQL injection.

മലയാളത്തിൽ

List-ൽ വെച്ച data server restart ചെയ്താൽ പോകും, അതിനാൽ SQLite database-ലേക്ക് മാറ്റുന്നു. വായിക്കാൻ (GET) ആർക്കും പറ്റും, പക്ഷേ ചേർക്കാനും മാറ്റാനും കളയാനും token വേണം, hostel-ൽ visitors-ന് സംസാരിക്കാം, പക്ഷേ അകത്ത് കയറാൻ ID card വേണം എന്നതു പോലെ. Pagination: 1000 students ഉണ്ടെങ്കിൽ ഒറ്റയടിക്ക് അയക്കാതെ 10 വീതം പേജുകളായി അയക്കുന്നു.

EXAMPLE: KEY PARTS OF THE STUDENT API

```

from flask import Flask, request, jsonify, g
import sqlite3
from functools import wraps

app = Flask(__name__)
app.config.update(DATABASE="students.db", API_TOKEN="mysecrettoken123")
app.json.sort_keys = False          # keep id, name, course order (Flask 2.3+)

def get_db():
    db = getattr(g, "_db", None)
    if db is None:
        db = g._db = sqlite3.connect(app.config["DATABASE"])
        db.row_factory = sqlite3.Row
    return db

@app.teardown_appcontext
def close_db(exc):
    db = getattr(g, "_db", None)
    if db is not None:
        db.close()

with sqlite3.connect(app.config["DATABASE"]) as conn:
    conn.execute("""CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY, name TEXT NOT NULL, course TEXT NOT NULL)""")

def require_token(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        token = request.headers.get("Authorization", "")
        if token != f"Bearer {app.config['API_TOKEN']}":
            return jsonify({"error": "Unauthorized"}), 401
        return f(*args, **kwargs)
    return wrapper

@app.route("/students", methods=["GET"])
def list_students():
    try:
        offset = int(request.args.get("offset", 0))
        limit = min(int(request.args.get("limit", 10)), 100)    # safety cap
    except ValueError:
        return jsonify({"error": "offset and limit must be integers"}), 400
    cur = get_db().cursor()
    cur.execute("SELECT id, name, course FROM students LIMIT ? OFFSET ?",
                (limit, offset))
    rows = [dict(r) for r in cur.fetchall()]
    return jsonify({"data": rows,
                    "pagination": {"offset": offset, "limit": limit, "count": len(rows)}})

@app.route("/students", methods=["POST"])
@require_token
def create_student():

```

```

payload = request.get_json(silent=True) or {}
err = validate_student_payload(payload, require_all=True)
if err:
    return jsonify({"error": err}), 400
db = get_db()
cur = db.cursor()
cur.execute("INSERT INTO students(name, course) VALUES(?, ?)",
            (payload["name"].strip(), payload["course"].strip()))
db.commit()
return jsonify({"id": cur.lastrowid, "name": payload["name"].strip(),
               "course": payload["course"].strip()}), 201

# GET /students/<int:sid>, PUT (merge) and DELETE follow the unit's code;
# DELETE checks cur.rowcount == 0 to return 404.

```

Validation helper (unit's Step 5): `validate_student_payload(payload, require_all)` returns an error string such as "Fields 'name' and 'course' are required", or None if the data is fine. The test run (the H header is {"Authorization": "Bearer mysecrettoken123"}):

```

show(c.post("/students", json={"name": "Aisha", "course": "Data Science"}))
show(c.post("/students", json={"name": "Aisha", "course": "Data Science"}, headers=H))
show(c.post("/students", json={"name": "Ravi"}, headers=H))
show(c.post("/students", json={"name": "Ravi", "course": "MCA"}, headers=H))
show(c.get("/students?offset=0&limit=2"))
show(c.get("/students?offset=1&limit=1"))
show(c.get("/students?limit=abc"))
show(c.put("/students/1", json={"course": "AI & ML"}, headers=H))
show(c.delete("/students/1", headers=H))
show(c.delete("/students/1", headers=H))

```

```

401 UNAUTHORIZED {"error":"Unauthorized"}
201 CREATED {"id":1,"name":"Aisha","course":"Data Science"}
400 BAD REQUEST {"error":"Fields 'name' and 'course' are required"}
201 CREATED {"id":2,"name":"Ravi","course":"MCA"}
200 OK {"data":[{"id":1,"name":"Aisha","course":"Data Science"},
{"id":2,"name":"Ravi","course":"MCA"}],"pagination":
{"offset":0,"limit":2,"count":2}}
200 OK {"data":[{"id":2,"name":"Ravi","course":"MCA"}],"pagination":
{"offset":1,"limit":1,"count":1}}
400 BAD REQUEST {"error":"offset and limit must be integers"}
200 OK {"id":1,"name":"Aisha","course":"AI & ML"}
200 OK {"message":"deleted"}
404 NOT FOUND {"error":"Resource not found"}

```

EXAMPLE: FETCH ALL BOOKS FROM SQLITE (CHECK YOUR PROGRESS 3, Q6)

```
@app.route('/db/books', methods=['GET'])
def get_books_db():
    cursor = get_db().cursor()
    cursor.execute("SELECT id, title FROM books")
    rows = cursor.fetchall()
    return jsonify([{"id": row[0], "title": row[1]} for row in rows])
```

```
200 OK [{"id":1,"title":"Python Basics"}, {"id":2,"title":"Flask for Beginners"},
{"id":3,"title":"Machine Learning in Action"}]
```

With a database the data is persistent, SQL allows filtering, searching and joins, and it scales to large data.

Exam tip: The unit sets `JSON_SORT_KEYS=False`. That config key was removed in Flask 2.3, so in current Flask it is silently ignored and the output comes out as `{"course": "Data Science", "id": 1, "name": "Aisha"}` (keys sorted), not in the order the unit shows. Use `app.json.sort_keys = False`. Also hard-coding the token is for teaching only; in production, read it from an environment variable.

What is FastAPI? Your first FastAPI app

FastAPI: a modern, high-performance Python web framework for building APIs, built on ASGI, that uses type hints for automatic validation and documentation.

ASGI (Asynchronous Server Gateway Interface): the standard interface between Python web servers and asynchronous (async) web applications.

FastAPI: APIs നിർമ്മിക്കാനുള്ള, ASGI അടിസ്ഥാനമാക്കിയ, ആധുനികവും വേഗതയുള്ളതുമായ Python web framework. Type hints ഉപയോഗിച്ച് validation-ഉം documentation-ഉം സ്വയം ഉണ്ടാക്കുന്നു.

ASGI (Asynchronous Server Gateway Interface): Python web servers-ഉം asynchronous (async) web applications-ഉം തമ്മിലുള്ള standard interface.

FastAPI is built on **Starlette** (web parts) and **Pydantic** (data validation). It is popular for four reasons: speed (in development and execution), automatic interactive documentation (Swagger UI at `/docs`, ReDoc at `/redoc`), type safety via Python type hints, and automatic validation with Pydantic models. Flask is traditionally WSGI (synchronous); FastAPI is ASGI and supports `async def`.

മലയാളത്തിൽ

FastAPI-യെ Flask-ന്റെ ഇളയ സഹോദരൻ എന്ന് പറയാം: കൂടുതൽ വേഗം, കൂടുതൽ സൗകര്യം. Function-ന്റെ parameter-ന് `book_id: int` എന്ന് type എഴുതിയാൽ മതി, തെറ്റായ data വന്നാൽ FastAPI തന്നെ error തരും. കൂടാതെ API-യുടെ documentation page ഒന്നും എഴുതാതെ തന്നെ `/docs`-ൽ തയ്യാറായിരിക്കും.

EXAMPLE

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to FastAPI!"}
```

Save as `main.py` and run `uvicorn main:app --reload` (file name `main`, object `app`; `--reload` restarts on code change). It serves at `http://127.0.0.1:8000/`. Tested with FastAPI's `TestClient`, including the two docs pages:

```
from fastapi.testclient import TestClient
c = TestClient(app)
r = c.get("/")
print(r.status_code, r.text)
print(c.get("/docs").status_code, c.get("/redoc").status_code)
```

```
200 {"message": "Welcome to FastAPI!"}
200 200
```

A returned dict is converted to JSON automatically; no `jsonify` is needed.

Exam tip: Q "Which packages are needed for FastAPI?": `fastapi` (the framework) and `uvicorn` (the ASGI server that runs it). The unit's answer says "Python 3.6+"; that is outdated, and current FastAPI releases need Python 3.10 or newer.

FastAPI routes with Pydantic models

Pydantic model: a class inheriting from `BaseModel` whose type-annotated fields define the structure of request or response data; FastAPI uses it to parse and validate JSON automatically.

Pydantic model: `BaseModel`-ൽ നിന്ന് inherit ചെയ്യുന്ന class; അതിലെ type-annotated fields request/response data-യുടെ ഘടന നിർവ്വചിക്കുന്നു. FastAPI ഇത് ഉപയോഗിച്ച് JSON സ്വയം parse ചെയ്ത് validate ചെയ്യുന്നു.

FastAPI has one decorator per method: `@app.get`, `@app.post`, `@app.put`, `@app.delete`. Path parameters are written `{book_id}` and typed in the function signature. A parameter typed as a Pydantic model is read from the JSON body. If the body does not match, FastAPI returns **422 Unprocessable Entity** with details, without running your function.

മലയാളത്തിൽ

Pydantic model ഒരു application form-ന്റെ template പോലെയാണ്: ഏതൊക്കെ കോളം വേണം, ഓരോന്നിലും എന്ത് തരം data വേണം എന്ന് അത് പറയുന്നു. Age എന്ന കോളത്തിൽ "many" എന്ന് എഴുതിയാൽ form അപ്പോൾ തന്നെ തിരിച്ചു വരും (422). "7" എന്ന string ആണെങ്കിൽ Pydantic അതിനെ int 7 ആക്കി സ്വീകരിക്കും.

EXAMPLE 1: BOOK API IN FASTAPI

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
books = []

class Book(BaseModel):
    id: int
    title: str

@app.post("/books")
def add_book(book: Book):
    books.append(book)
    return {"message": "Book added successfully"}

@app.get("/books")
def get_books():
    return books

@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    for book in books:
        if book.id == book_id:
            books.remove(book)
            return {"message": "Book deleted"}
    return {"error": "Book not found"}
```

Calls: POST {"id": 1, "title": "Learning FastAPI"}, GET, DELETE /books/1 twice, then POST {"id": "one", ...}:

```
200 {"message":"Book added successfully"}
200 [{"id":1,"title":"Learning FastAPI"}]
200 {"message":"Book deleted"}
200 {"error":"Book not found"}
422 {"detail":[{"type":"int_parsing","loc":["body","id"],"msg":"Input should be a valid integer, unable to parse string as an integer","input":"one"}]}
```

Note the fourth line: "Book not found" arrives with status **200**, which is wrong for REST. The fix is to raise `HTTPException`:

```

from fastapi import HTTPException

@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    for book in books:
        if book.id == book_id:
            books.remove(book)
            return {"message": "Book deleted"}
    raise HTTPException(status_code=404, detail="Book not found")

```

```
404 {"detail": "Book not found"}
```

EXAMPLE 2: @APP.GET VS @APP.POST AND A VALIDATION ERROR (CHECK YOUR PROGRESS 3, Q3 AND Q4)

```

@app.get("/hello")
def say_hello():
    return {"message": "Hello, world!"}

@app.post("/submit")
def receive_data(data: dict):
    return {"received": data}

class Book(BaseModel):
    title: str
    author: str
    pages: int

@app.post("/addbook")
def add_book(book: Book):
    return {"message": "Book received", "title": book.title}

```

GET /hello; POST /submit with {"course": "MCS-082"}; POST /addbook with "pages": "many", then with "pages": 1100:

```

200 {"message": "Hello, world!"}
200 {"received": {"course": "MCS-082"}}
422 {"detail": [{"type": "int_parsing", "loc": ["body", "pages"], "msg": "Input should be a valid integer, unable to parse string as an integer", "input": "many"}]}
200 {"message": "Book received", "title": "AI"}

```

Exam tip: In FastAPI, POST returns 200 by default. For a proper "Created" response write `@app.post("/books", status_code=201)`. Validation failures in FastAPI are 422, not 400.

Protecting a FastAPI endpoint

HTTPException: the FastAPI exception that, when raised, stops the handler and sends an error response with the given status code and detail message.

HTTPException: raise ചെയ്താൽ handler അവിടെ നിർത്തി, കൊടുത്ത status code-ഉം detail message-ഉം ഉള്ള error response അയക്കുന്ന FastAPI exception.

A function parameter that is not in the path and not a model (like `token: str = ""`) is read from the query string, so `/secure?token=admin123` passes `token="admin123"`. The unit's version returns an error dict, which again goes out with status 200. Raising `HTTPException(401)` is correct.

മലയാളത്തിൽ

"Access Denied" എന്ന് എഴുതിയ കത്ത് "OK" എന്ന seal വെച്ച് അയക്കുന്നത് പോലെയാണ് 200 status-ൽ error അയക്കുന്നത്. Client program status code നോക്കിയാണ് തീരുമാനം എടുക്കുന്നത്, അതിനാൽ ശരിയായ code (401) തന്നെ അയക്കണം.

EXAMPLE

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/secure")
def secure_route(token: str = ""):
    if token != "admin123":
        raise HTTPException(status_code=401, detail="Access Denied")
    return {"message": "Welcome Admin!"}
```

`/secure?token=admin123` and `/secure?token=wrong` (the unit's version gave 200 `{"error": "Access Denied"}` for the second):

```
200 {"message": "Welcome Admin!"}
401 {"detail": "Access Denied"}
```

Exam tip: Tokens in the URL end up in browser history and server logs. Real APIs send them in the Authorization header (OAuth2 / JWT). The unit's snippet also imports Request without using it.

Advanced FastAPI: async, dependencies and background tasks

Dependency (Depends): a function that FastAPI runs before the endpoint and injects its result, used for shared logic such as authentication.

Background task: work added with `BackgroundTasks.add_task()` that runs after the response is sent, so the client does not wait.

Dependency (Depends): endpoint-ന് മുമ്പ് FastAPI run ചെയ്ത് അതിന്റെ result inject ചെയ്യുന്ന function; authentication പോലുള്ള പൊതു logic-ന് ഉപയോഗിക്കുന്നു.

Background task: `BackgroundTasks.add_task()` ഉപയോഗിച്ച് ചേർക്കുന്ന ജോലി; response അയച്ച ശേഷം run ആകുന്നതിനാൽ client കാത്തിരിക്കേണ്ടതില്ല.

The Library Catalog API shows the advanced features: `async def` endpoints (with `await asyncio.sleep()` standing in for slow DB calls), Pydantic Field constraints (`min_length`, `ge`, `le`), a nested model (Author inside Book), Query parameters for search, paging and sorting, `response_model`, an API-key dependency and a background task after create.

മലയാളത്തിൽ

Depends എന്നത് ഓഫീസിലെ security guard പോലെ: ഓരോ മുറിയിലും (endpoint) ID check ചെയ്യാൻ വേറെ വേറെ ആളെ വെക്കാതെ, ഒരാളെ gate-ൽ നിർത്തുന്നു.

Background task: Akshaya centre-ൽ അപേക്ഷ വാങ്ങി receipt ഉടൻ തരും (response), processing പിന്നീട് നടക്കും. `async` ആയതിനാൽ ഒരു request DB-ക്കായി കാത്തിരിക്കുമ്പോൾ server മറ്റ് requests കൈകാര്യം ചെയ്യും.

EXAMPLE: MODELS AND KEY ROUTES

```

import asyncio
from typing import List, Optional
from fastapi import FastAPI, HTTPException, Query, BackgroundTasks, Depends, status
from pydantic import BaseModel, Field

app = FastAPI(title="Library Catalog API", version="1.0")

class Author(BaseModel):
    name: str = Field(..., min_length=2, max_length=80)
    country: Optional[str] = Field(default=None, max_length=80)

class Book(BaseModel):
    id: int = Field(..., ge=1)
    title: str = Field(..., min_length=3, max_length=120)
    year: int = Field(..., ge=1500, le=2100)
    author: Author

class BookUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=3, max_length=120)
    year: Optional[int] = Field(default=None, ge=1500, le=2100)
    author: Optional[Author] = None

BOOKS: List[Book] = []

def require_api_key(x_api_key: Optional[str] = Query(default=None, alias="api_key")):
    if x_api_key != "demo-key":
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                             detail="Invalid API key")
    return True

async def index_book_background(book: Book):
    await asyncio.sleep(0.1)      # simulate indexing / notification

@app.get("/books", response_model=List[Book])
async def list_books(keyword: Optional[str] = Query(default=None),
                    skip: int = Query(default=0, ge=0),
                    limit: int = Query(default=10, ge=1, le=100),
                    sort: Optional[str] = Query(default="title")):
    items = BOOKS
    if keyword:
        k = keyword.lower()
        items = [b for b in items if k in b.title.lower() or k in b.author.name.lower()]
    key = (lambda b: b.year) if sort == "year" else (lambda b: b.title.lower())
    return sorted(items, key=key)[skip: skip + limit]

@app.post("/books", response_model=Book, status_code=201)
async def create_book(book: Book, bg: BackgroundTasks,
                    ok: bool = Depends(require_api_key)):
    if any(b.id == book.id for b in BOOKS):
        raise HTTPException(status_code=400, detail="ID already exists")
    BOOKS.append(book)

```

```

bg.add_task(index_book_background, book)
return book

@app.put("/books/{book_id}", response_model=Book)
async def update_book(book_id: int, patch: BookUpdate,
                      ok: bool = Depends(require_api_key)):
    for i, b in enumerate(BOOKS):
        if b.id == book_id:
            updated = Book.model_validate(**b.model_dump(),
                                         **patch.model_dump(exclude_unset=True))

            BOOKS[i] = updated
            return updated
    raise HTTPException(status_code=404, detail="Book not found")

# GET /books/{book_id} and DELETE /books/{book_id} raise 404 the same way.

```

Test calls (in order): create Deep Learning with the key; create without the key; create it again; create Artificial Intelligence; search ?keyword=deep&skip=0&limit=5&sort=year; list ?sort=year; ?limit=500; PUT {"year": 2018}; delete; get.

```

201 {"id":101,"title":"Deep Learning","year":2016,"author":{"name":"Ian
Goodfellow","country":"USA"}}
401 {"detail":"Invalid API key"}
400 {"detail":"ID already exists"}
201 {"id":102,"title":"Artificial Intelligence","year":1995,"author":
{"name":"Stuart Russell","country":null}}
200 [{"id":101,"title":"Deep Learning","year":2016,"author":{"name":"Ian
Goodfellow","country":"USA"}}]
200 [{"id":102,"title":"Artificial Intelligence","year":1995,"author":
{"name":"Stuart Russell","country":null}},{"id":101,"title":"Deep
Learning","year":2016,"author":{"name":"Ian Goodfellow","country":"USA"}}]
422 {"detail":[{"type":"less_than_equal","loc":["query","limit"],"msg":"Input
should be less than or equal to 100","input":"500","ctx":{"le":100}}]}
200 {"id":101,"title":"Deep Learning","year":2018,"author":{"name":"Ian
Goodfellow","country":"USA"}}
200 {"message":"deleted"}
404 {"detail":"Book not found"}

```

Posting {"id": 102, "title": "AI", "year": 1400, "author": {"name": "R"}} and printing each error's loc and msg shows that Pydantic checks every field, including the nested one:

```

422
['body', 'title'] - String should have at least 3 characters
['body', 'year'] - Input should be greater than or equal to 1500
['body', 'author', 'name'] - String should have at least 2 characters

```

Exam tip: The unit's update line `b.copy(update=patch.dict(exclude_unset=True))` is Pydantic v1 style. In Pydantic v2 (current), `.copy()` and `.dict()` are deprecated (use `model_copy`, `model_dump`), and `copy(update=...)` does not re-validate. Tested: updating the author to `{"name": "Yoshua Bengio"}` stored a raw dict and returned `"author": {"name": "Yoshua Bengio"}` with no country field. Rebuilding with `Book.model_validate({...})`, as above, gives a proper Author with `"country": null`. For a duplicate ID, 409 Conflict is more precise than the unit's 400.

Flask vs FastAPI

Both build RESTful APIs that exchange JSON. Choose Flask for small, simple or learning projects and when you want full control. Choose FastAPI when you want automatic validation and docs, type safety, and high concurrency with async code.

മലയാളത്തിൽ

Flask ഒരു സാധാരണ സൈക്കിൾ പോലെ: ലളിതം, എല്ലാം നമ്മൾ തന്നെ ചവിട്ടണം. FastAPI ഒരു gear ഉള്ള സൈക്കിൾ: validation, docs, async എന്നിവ ആദ്യം മുതലേ കൂടെയുണ്ട്. ചെറിയ project-ന് Flask, വലിയ production API-ക്ക് FastAPI എന്ന് ഓർക്കാം.

Feature	Flask	FastAPI
Interface	WSGI (synchronous)	ASGI (async supported)
Route decorator	<code>@app.route('/x', methods=['GET'])</code>	<code>@app.get('/x')</code>
Path parameter	<code>/books/<int:id></code>	<code>/books/{id} + id: int</code>
Read JSON body	<code>request.get_json()</code>	Pydantic model parameter
Return JSON	<code>jsonify(...)</code> (dicts also auto-converted)	Return dict / model directly
Validation	Manual	Automatic (Pydantic), 422 on failure
Error response	<code>return ..., 404 / @app.errorhandler</code>	<code>raise HTTPException(404)</code>
API docs	Needs extensions	Built in: <code>/docs</code> , <code>/redoc</code>
Run	<code>app.run()</code> / <code>flask run</code> (port 5000)	<code>uvicorn main:app --reload</code> (port 8000)
Testing	<code>app.test_client()</code>	<code>TestClient(app)</code>

Quick recap

- An API is a set of rules letting two applications communicate; a Web API does it over HTTP (client = customer, API = waiter, server = kitchen).
- REST (Roy Fielding, 2000) is an architectural style; IGNOU's six constraints: stateless, client-server, uniform interface, resource-based, representations, cacheable.
- A request = method + URL + optional headers + optional body. A response = status code + headers + body.
- CRUD ↔ HTTP: Create = POST, Read = GET, Update = PUT (full) / PATCH (partial), Delete = DELETE.
- Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 405 Method Not Allowed, 422 validation error, 500 server error.
- JSON: `json.dumps` (Python → JSON string), `json.loads` (JSON → Python); `true/null` ↔ `True/None`.
- Flask: `@app.route(path, methods=[...])`, `<int:id>`, `request.get_json()`, `request.args`, `jsonify()`, `return (body, code)`.
- Validate input and return 400; use `@app.errorhandler` for consistent JSON errors that hide internal details.
- Use a database (SQLite) instead of a list for persistence; use `?` placeholders, per-request connections in `g`, and pagination with `offset/limit`.

- Bearer token: header `Authorization: Bearer <token>`; missing or wrong token → 401.
- FastAPI = Starlette + Pydantic on ASGI; run with `uvicorn main:app --reload`; docs at `/docs` and `/redoc`.
- Pydantic models validate request bodies automatically; raise `HTTPException` for errors instead of returning an error dict with 200.
- Depends injects shared logic such as auth; `BackgroundTasks` runs work after the response.
- Test without a browser: `app.test_client()` (Flask), `TestClient(app)` (FastAPI), `requests`, or `curl`.

Check yourself

Q. 1. Match each operation with its HTTP method: (a) retrieve a resource (b) create a new resource (c) update an existing resource (d) delete a resource.

(a) GET, (b) POST, (c) PUT (PATCH for a partial update), (d) DELETE.

Q. 2. Describe the structure of an HTTP request with an example.

Four parts: **method** (the action, e.g. POST), **URL** (the resource, e.g. `/users`), **headers** (metadata, e.g. `Content-Type: application/json`, `Authorization: Bearer abc123`) and **body** (data, used with POST/PUT/PATCH, e.g. `{"name": "Alice", "email": "alice@example.com"}`). Together: "POST `/users` with a JSON body creates user Alice".

Q. 3. A weather app shows Delhi's weather. Explain how HTTP methods, JSON and client-server interaction work together.

(1) The app (client) sends `GET /weather?city=Delhi`. (2) The server looks up the data in its database or another source. (3) It replies 200 OK with a JSON body such as `{"city": "Delhi", "temperature": "32°C", "status": "Sunny"}`. (4) The app parses the JSON and displays it. GET is used because the app only reads data; client and server are separate and each request is stateless.

Q. 4. Critically analyse statelessness in REST. What goes wrong if an API is stateful?

Stateless: each request carries all needed information, and the server keeps no session. This gives scalability (any server can handle any request), reliability and easy load balancing. A stateful API must keep session data for every client, which uses memory, loses sessions on a crash and breaks when a load balancer sends the next request to another server. For example, a railway booking half-done on server A fails when the payment step reaches server B.

Q. 5. What happens when a request is sent to an undefined endpoint such as /unknown in Flask? What if the URL exists but the method is wrong?

Undefined URL → **404 Not Found** (Flask's HTML page, or your JSON if you registered `@app.errorhandler(404)`). URL exists but method is not allowed (e.g. DELETE /books in the Book API) → **405 Method Not Allowed**. Both were verified with the test client in topics 10 and 12.

Q. 6. Write a Flask endpoint that returns the square of a number sent as a query parameter: /square?num=5 → {"result": 25}.

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route('/square', methods=['GET'])
def calculate_square():
    num = request.args.get('num', default=0, type=int)
    return jsonify({"result": num * num})
```

```
200 OK {"result":25}
```

Note: the unit's printed answer has a missing quote in ' /square and curly quotes, so it will not run as printed.

Q. 7. Your Book API stores data in a Python list. State the limitations and how a database fixes them.

Limitations: data is lost when the server restarts; no querying (search, filter, join); poor with large data; not safe when several processes run. Using SQLite: data is persistent, SQL gives WHERE/LIKE/joins, and it scales. The /db/books endpoint in topic 13 returned all three rows from `books.db` as JSON by converting each row to a dict.

Q. 8. What is FastAPI and why is it popular? Which packages does it need?

FastAPI is a modern, high-performance Python framework for building APIs, built on ASGI (Starlette + Pydantic). Popular because of speed, automatic Swagger/ReDoc docs, type safety from type hints, automatic validation, and async support. Install `fastapi` (framework) and `uvicorn` (ASGI server): `pip install fastapi "uvicorn[standard]"`.

Q. 9. What is a Pydantic model? What does FastAPI do if the client sends "pages": "many" for a field declared pages: int?

A Pydantic model is a `BaseModel` subclass whose typed fields describe the expected data. FastAPI validates the request body against it, converts JSON to a Python object, and on failure returns **422** without calling the function: `{"detail":[{"type":"int_parsing","loc":["body","pages"],"msg":"Input should be a valid integer, ...","input":"many"}]}`. A numeric string like "7" is accepted and converted to 7.

Q. 10. Design a Flask Student Records API (GET all, GET one, POST, PUT, DELETE) with id, name, course, and show example responses.

```
from flask import Flask, request, jsonify
app = Flask(__name__)
students = [{"id": 1, "name": "Alice", "course": "Computer Science"},
            {"id": 2, "name": "Bob", "course": "Mathematics"}]

@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)

@app.route('/students/<int:student_id>', methods=['GET'])
def get_student(student_id):
    for s in students:
        if s['id'] == student_id:
            return jsonify(s)
    return jsonify({"error": "Student not found"}), 404

@app.route('/students', methods=['POST'])
def add_student():
    new = request.get_json(silent=True)
    if not new or not all(k in new for k in ("id", "name", "course")):
        return jsonify({"error": "Invalid data"}), 400
    students.append(new)
    return jsonify({"message": "Student added successfully"}), 201

@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    data = request.get_json(silent=True) or {}
    for s in students:
        if s['id'] == student_id:
            s.update(data)
            return jsonify(s)
    return jsonify({"error": "Student not found"}), 404

@app.route('/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    for s in students:
        if s['id'] == student_id:
            students.remove(s)
            return jsonify({"message": "Student deleted"})
    return jsonify({"error": "Student not found"}), 404
```

GET /students, GET /students/1, POST Charlie/Physics, PUT {"course": "Data Science"} on 3, DELETE 3, POST {"name": "X"} (Flask 3 sorts keys by default):

```
200 OK [{"course":"Computer Science","id":1,"name":"Alice"},
{"course":"Mathematics","id":2,"name":"Bob"}]
200 OK {"course":"Computer Science","id":1,"name":"Alice"}
201 CREATED {"message":"Student added successfully"}
200 OK {"course":"Data Science","id":3,"name":"Charlie"}
200 OK {"message":"Student deleted"}
400 BAD REQUEST {"error":"Invalid data"}
```

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.